AI Fundamentals Explained in Fifty Papers

Jonathan Y. Ma

Contents

1 The Big Picture **3**
- 1.1 Computing Machinery and Intelligence . . . 3
- 1.2 Deep Learning . . . 6
- 1.3 Is It Enough to Get the Behaviour Right? . . . 9
- 1.4 Synthesis: Behavior, Representation, and General Competence . . . 11

2 First Networks **12**
- 2.1 The Perceptron . . . 12
- 2.2 Learning Representations by Back-Propagating Errors . . . 14
- 2.3 Backpropagation Applied to Handwritten Zip Code Recognition . . . 15
- 2.4 Long Short-Term Memory . . . 17
- 2.5 A Deep Probabilistic Language Model . . . 19
- 2.6 Synthesis: The Emergence of Representation Learning . . . 21

3 Words as Numbers **21**
- 3.1 Efficient Estimation of Word Representations in Vector Space . . . 21

4 Going Deep **25**
- 4.1 Deep Sparse Rectifier Neural Networks . . . 25
- 4.2 Dropout: A Simple Way to Prevent Neural Networks from Overfitting . . . 26
- 4.3 ImageNet Classification with Deep Convolutional Neural Networks . . . 27
- 4.4 Adam: A Method for Stochastic Optimization . . . 28
- 4.5 Batch Normalization . . . 29
- 4.6 Deep Residual Learning for Image Recognition . . . 30
- 4.7 Synthesis: Making Depth Trainable . . . 32

5 From Translation to Transformers **32**
- 5.1 Sequence to Sequence Learning with Neural Networks . . . 32
- 5.2 Neural Machine Translation by Jointly Learning to Align and Translate . . . 34
- 5.3 Neural Machine Translation of Rare Words with Subword Units . . . 35
- 5.4 Attention Is All You Need . . . 36
- 5.5 Synthesis: From Compression to Addressable Context . . . 38

6 The First Pre-Trained Transformers **38**
- 6.1 Improving Language Understanding by Generative Pre-Training . . . 38
- 6.2 BERT: Pre-Training for Language Understanding . . . 40
- 6.3 Language Models Are Unsupervised Multitask Learners . . . 41
- 6.4 Synthesis: Three Transfer Regimes . . . 42

7 Scaling Up **43**
- 7.1 The Bitter Lesson . . . 43
- 7.2 Language Models Are Few-Shot Learners . . . 44
- 7.3 Scaling Laws for Neural Language Models . . . 45
- 7.4 Training Compute-Optimal Large Language Models . . . 47

7.5 Synthesis: Scaling as an Experimental Science 48

8 Deep Reinforcement Learning **48**
8.1 Human-Level Control through Deep Reinforcement Learning 48
8.2 Mastering the Game of Go with Deep Neural Networks and Tree Search . . . 49
8.3 Deep Reinforcement Learning from Human Preferences 51
8.4 Synthesis: Learning Values, Planning, and Learning the Objective 52

9 From Model to Assistant **53**
9.1 Training Language Models to Follow Instructions with Human Feedback . . 53
9.2 Direct Preference Optimization: Your Language Model Is a Reward Model . 54
9.3 Chain-of-Thought Prompting Elicits Reasoning in Large Language Models . 55
9.4 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks 55
9.5 ReAct: Synergizing Reasoning and Acting in Language Models 56
9.6 Why Language Models Hallucinate 57
9.7 Synthesis: From a Language Model to an Assistant 58

10 Reasoning Models **58**
10.1 DeepSeekMath 58
10.2 DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL 59
10.3 s1: Simple Test-Time Scaling 61
10.4 Synthesis: Three Axes of Reasoning-System Scaling 62

11 Images, Generation, and Science **62**
11.1 Generative Adversarial Nets 62
11.2 Denoising Diffusion Probabilistic Models 63
11.3 High-Resolution Image Synthesis with Latent Diffusion Models 64
11.4 An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale 65
11.5 Learning Transferable Visual Models from Natural Language Supervision . . 65
11.6 Highly Accurate Protein Structure Prediction with AlphaFold 66
11.7 Synthesis: Representations Determine the Feasible Computation 67

12 Under the Hood **67**
12.1 Distilling the Knowledge in a Neural Network 67
12.2 Quantization and Training of Neural Networks 68
12.3 LoRA: Low-Rank Adaptation of Large Language Models 69
12.4 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness 70
12.5 Efficient Memory Management for LLM Serving with PagedAttention 70
12.6 Switch Transformers: Scaling to Trillion Parameter Models 71
12.7 Synthesis: Six Different Meanings of Efficiency 72

1 The Big Picture

1.1 Computing Machinery and Intelligence (Link)

Purpose and argumentative strategy

Turing does not try to define either "machine" or "thinking." He replaces the metaphysical question *Can machines think?* with an operational one: can a machine sustain a text-only conversation well enough that an interrogator cannot reliably distinguish it from a person? This is a methodological proposal. An observable test makes competing claims answerable by evidence even when their theoretical vocabulary remains contested.

The original imitation game first asks an interrogator to identify a man and a woman who communicate through a teleprinter. Turing then replaces one participant by a machine. Text communication deliberately removes appearance, voice, and physical dexterity, concentrating the experiment on capacities that can be expressed in linguistic interaction. The paper therefore proposes an *epistemic criterion* for attributing intelligence, not a definition of consciousness and not a claim that language is the only possible form of intelligence.

A statistical formulation of the imitation game

Let an interrogator I interact for T rounds with either a human system H or a machine M. The interaction induces transcript distributions P_H^I and P_M^I on a transcript space $\mathcal{T}_T$. If the prior odds are equal, the best possible discriminator has success probability

$$p^*_{\mathrm{succ}}(I) = \frac{1}{2}\left(1 + \mathrm{TV}(P_H^I, P_M^I)\right),$$

where

$$\mathrm{TV}(P, Q) = \sup_{A \subseteq \mathcal{T}_T} |P(A) - Q(A)| = \frac{1}{2} \sum_{\tau \in \mathcal{T}_T} |P(\tau) - Q(\tau)|$$

in the discrete case.

Proposition 1.1 (Indistinguishability is distributional)**.** *If $P_H^I = P_M^I$, no decision rule based only on the transcript can distinguish the two systems with probability greater than $1/2$. More generally, its advantage over random guessing is at most $\mathrm{TV}(P_H^I, P_M^I)/2$.*

Proof. For a decision region A in which the rule declares "human," its success probability is

$$\frac{1}{2}P_H^I(A) + \frac{1}{2}P_M^I(A^c) = \frac{1}{2} + \frac{1}{2}\{P_H^I(A) - P_M^I(A)\}.$$

Maximizing over A gives the total-variation expression above. □

This modern formalization exposes three variables that the informal phrase "passes the Turing test" suppresses: the population of interrogators, the allowed interaction protocol, and the time horizon T. Passing a short, scripted exchange is weaker evidence than succeeding under adaptive, open-ended questioning.

Digital computers and universality

Turing describes a digital computer as having three functional parts:

- a *store* containing data and instructions;
- an *executive unit* carrying out elementary operations; and
- a *control* that selects instructions in the prescribed order.

Its discrete states may approximate a continuous physical process to arbitrary precision, subject to finite storage and time. The crucial fact is *universality*: a universal digital computer can simulate any other discrete-state machine once supplied with an encoding of that machine and its program. Consequently the central empirical issue is not whether one must invent a special physical substance for thought, but whether suitable programs, memory, and computation can produce the relevant behavior.

This is a claim about computational capacity, not efficiency. A simulation may require prohibitive time or space. The distinction between computability and tractability becomes central in Levesque's later refinement of the behavioral argument.

The objections and what Turing's replies establish

Turing surveys objections rather than offering one deductive proof.

Theological and "heads in the sand" objections. These concern what humans wish to reserve for themselves, rather than observable restrictions on machines.

Mathematical limitations. Gödelian and related results imply that a sufficiently expressive formal system has true statements it cannot prove. Turing observes that humans are also fallible and that a limitation on one machine does not identify a question that every human can answer correctly. Incompleteness therefore blocks claims of universal infallibility, not the possibility of machine intelligence.

The argument from consciousness. First-person experience is not directly observable in another person either. Requiring access to subjective experience before crediting a machine would push us toward solipsism. The imitation game instead uses the public evidence by which we ordinarily judge other minds.

Arguments from disability. Lists asserting that a machine can never be kind, creative, humorous, adaptive, or capable of error usually generalize from existing machines. They are empirical challenges, not impossibility proofs.

Lady Lovelace's objection. A machine is said to originate nothing and merely execute orders. Turing answers that programs often surprise their creators; the consequences of rules need not be transparent to the person who specified them. Surprise alone is not proof of originality, but neither is explicit programming proof of its absence.

Continuity and informality. The continuous nervous system and the apparent absence of fixed rules governing human conduct do not imply that no discrete machine can match the system's externally visible responses. A discrete model can approximate continuous dynamics, and rules of operation should not be confused with simple laws of behavior visible to an observer.

Turing also discusses extrasensory perception because he treats it as an empirical claim then taken seriously by some investigators. It is not a substantive component of the computational thesis.

Learning machines

Rather than hand-code an adult mind, Turing proposes constructing a "child-machine" and educating it. He separates three ingredients:

$$\text{initial state,} \qquad \text{education,} \qquad \text{other experience.}$$

He compares the search over learning-machine designs to evolution: structure plays the role of hereditary material, changes resemble mutation, and the experimenter's evaluation supplies selection. Rewards and punishments can shape behavior, while a random component can make search less predictable and help explore alternatives.

In current notation, this anticipates choosing parameters θ by interaction with an environment:

$$\theta_{t+1} = \theta_t + \eta_t \widehat{g}_t,$$

where $\widehat{g}_t$ is an update inferred from examples, feedback, or reward. The formula is modern, but the conceptual move is Turing's: intelligence should be sought in a system that *acquires* competence, not only in a static list of adult rules.

What the paper does and does not show

Turing predicts that machines with roughly 10^9 binary digits of storage would make the imitation game meaningful and suggests that, by 2000, an average interrogator would have at most a 70% chance of correct identification after five minutes. This is a forecast, not the definition of a universal pass mark.

The paper's durable contribution is methodological: behavioral evidence can discipline a vague philosophical dispute, universal computation makes software and learning central, and objections should be turned into testable limitations. It does not prove that behavioral equivalence entails phenomenal consciousness, nor does success at one finite protocol establish unrestricted intelligence.

1.2 Deep Learning (Link)

Scope of the review

LeCun, Bengio, and Hinton survey the 2015 deep-learning paradigm rather than introducing one new algorithm. Their organizing thesis is that useful representations can be learned from data. Instead of fixing a hand-engineered feature map $\phi(x)$ and fitting only a shallow predictor, deep learning fits a composition

$$f_\theta(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x),$$

so successive layers transform raw input into increasingly task-relevant features. A representation is *distributed* when a concept is encoded by a pattern across multiple coordinates and each coordinate participates in multiple concepts. Such codes can reuse factors and generalize to combinations not explicitly observed during training.

Supervised learning as empirical risk minimization

For labeled data $\{(x_i, y_i)\}_{i=1}^n$, training approximately solves

$$\widehat{\theta} \in \arg\min_{\theta} \widehat{R}_n(\theta), \qquad \widehat{R}_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

A feed-forward layer has the form

$$z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}, \qquad h^{(\ell)} = \sigma_\ell(z^{(\ell)}),$$

with $h^{(0)} = x$. The review emphasizes rectified linear units $\sigma(z) = \max\{0, z\}$, which avoid the strong saturation of sigmoid units on their active side and made deep supervised networks easier to optimize.

For K-class classification, logits $a \in \mathbb{R}^K$ give

$$p_\theta(y = k \mid x) = \frac{e^{a_k}}{\sum_{j=1}^{K} e^{a_j}}, \qquad \ell(a, y) = -\log p_\theta(y \mid x).$$

Mini-batch stochastic gradient descent estimates the full gradient using a small subset B_t:

$$\theta_{t+1} = \theta_t - eta_t \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_\theta \ell(f_\theta(x_i), y_i).$$

Backpropagation is reverse-mode differentiation

Backpropagation efficiently applies the chain rule to a computational graph. For one example, let $\delta^{(\ell)} = \partial \ell / \partial z^{(\ell)}$.

Proposition 1.2 (Backpropagation recursion)**.** *For the feed-forward network above,*

$$\delta^{(L)} = \nabla_{h^{(L)}} \ell \odot \sigma_L'(z^{(L)}), \qquad \delta^{(\ell)} = \left(W^{(\ell+1)\top} \delta^{(\ell+1)} \right) \odot \sigma_\ell'(z^{(\ell)}),$$

and

$$\nabla_{W^{(\ell)}} \ell = \delta^{(\ell)} h^{(\ell-1)\top}, \qquad \nabla_{b^{(\ell)}} \ell = \delta^{(\ell)}.$$

Proof. The Jacobian of $z^{(\ell+1)}$ with respect to $h^{(\ell)}$ is $W^{(\ell+1)}$, while the elementwise nonlinearity contributes $\operatorname{diag}(\sigma_\ell'(z^{(\ell)}))$. Applying the chain rule backward gives the recursion. Since $z_j^{(\ell)} = \sum_k W_{jk}^{(\ell)} h_k^{(\ell-1)} + b_j^{(\ell)}$, differentiating with respect to each weight gives the outer product. □

The computation reuses intermediate activations and adjoints, so its cost is a small constant multiple of the forward evaluation. Backpropagation supplies a gradient; it does not by itself guarantee that a nonconvex training problem has reached a global optimum. The authors stress that large networks nonetheless often reach useful solutions and that poor local minima appeared less damaging in practice than had been feared.

Convolutional networks: symmetry through parameter sharing

A discrete one-dimensional cross-correlation (usually called convolution in deep learning) is

$$(k * x)[u] = \sum_v k[v]x[u+v].$$

Local receptive fields exploit spatial locality; using the same kernel at every position reduces the parameter count and encodes translation symmetry.

Proposition 1.3 (Translation equivariance)**.** *Let* $(T_a x)[u] = x[u-a]$*. Ignoring boundary effects,*

$$k * (T_a x) = T_a(k * x).$$

Proof. Direct substitution gives $[k * (T_a x)](u) = \sum_v k[v]x[u+v-a] = (k * x)(u-a)$. □

Equivariance means that translating the input translates the feature map; it is not invariance. Pooling or spatial aggregation can make later representations less sensitive to small translations and distortions. Stacking convolution, nonlinearity, and pooling builds features ranging from edges to motifs to object parts. The review attributes the resurgence of computer vision to this architecture together with larger labeled data sets and efficient GPU training, not to depth in isolation.

Recurrent networks and gradients through time

For a sequence, a simple recurrent network updates

$$h_t = \phi(Wh_{t-1} + Ux_t + b), \qquad y_t = g(Vh_t).$$

Unrolling it in time produces a deep network whose copies share parameters. With total loss $L = \sum_t \ell_t$, the dependence of a later state on an earlier one contains a Jacobian product

$$\frac{\partial h_t}{\partial h_s} = J_t J_{t-1} \cdots J_{s+1}, \qquad J_j = \operatorname{diag}(\phi'(a_j))W.$$

Thus

$$\left\| \frac{\partial h_t}{\partial h_s} \right\| \le \prod_{j=s+1}^{t} \|J_j\|.$$

If typical factors have norm below one, gradients vanish exponentially; if they exceed one, gradients may explode. Gated architectures such as LSTMs create paths along which state and gradients can persist, using learned gates to decide what to store, expose, and forget.

The review connects recurrent encoders and decoders to language modeling, machine translation, speech recognition, and image captioning. Learned word vectors place semantically or syntactically related items near one another and can expose approximately linear relations, illustrating how distributed representations support transfer across combinations.

Unsupervised learning and the 2015 research frontier

The authors use *unsupervised learning* broadly for extracting structure from unlabeled data. Autoencoders, generative models, and predictive objectives can learn useful internal variables before supervised fine-tuning. This mattered especially when labels were scarce. They argue that unsupervised learning should become increasingly important because raw observations are abundant and human learning is not driven solely by explicit labels.

The paper also anticipates systems combining convolutional perception, recurrent memory, attention, and reinforcement learning. Its historical vantage point matters: it predates transformers, large-scale self-supervised language models, and modern foundation-model alignment. Consequently it should be read as a map of the ingredients and bottlenecks visible in 2015, not as a complete theory of present-day AI.

Theoretical cautions

Expressivity, optimization, and generalization are distinct questions. A network class may represent a target function without gradient descent finding it; a low training loss may coexist with poor population risk; and an architecture's inductive bias determines which solutions are accessible from finite data. The review provides strong empirical evidence for learned hierarchies but does not prove that depth universally discovers semantic abstractions. Interpretability, robustness under distribution shift, causal reasoning, and sample efficiency remain separate problems.

1.3 Is It Enough to Get the Behaviour Right? (Link)

Question and position

Levesque revisits the dispute between Turing's behavioral criterion and Searle's Chinese Room. His question is narrower than whether behavioral equivalence logically entails consciousness: under what conditions is behavior good evidence that a system possesses the competence that normally explains that behavior? His answer depends on the richness of the task and the computational resources needed to fake it.

In the Chinese Room, a person who knows no Chinese follows an English rulebook that maps Chinese inputs to Chinese outputs. Searle argues that correct external responses do not imply understanding. The standard "systems reply" attributes understanding to the entire person–rulebook system rather than to the person alone. Searle counters that the person could memorize the book and still fail to understand Chinese. Levesque argues that the nature and feasible size of this book cannot be ignored.

Two kinds of rulebook

Levesque separates two mechanisms that may have identical behavior on a fixed set of cases:

Type 1: extensional lookup. The book contains a separate response for each possible complete input. It can reproduce a finite table without representing a reusable relation among cases.

Type 2: intensional procedure. The book decomposes inputs, maintains intermediate state, and applies reusable rules. Its compactness comes from capturing structure shared across many instances.

Memorizing a Type 1 table need not confer the relevant competence. Memorizing and fluently applying a Type 2 procedure is much closer to learning the skill itself. The systems reply becomes persuasive only after asking which sort of book could actually generate the behavior.

The Summation Room

To remove distractions surrounding natural language, Levesque considers a room asked to add twenty ten-digit integers and return their twelve-digit sum. There are

$$(10^{10})^{20} = 10^{200}$$

possible ordered input lists before accounting for alternate surface forms. A Type 1 book with one entry per input is physically unrealizable. A short Type 2 book instead describes column-wise addition, carry propagation, and a finite table of elementary digit sums. One may change the chunk size—for example, add blocks in base 100—but any feasible rulebook exploits the compositional structure of addition.

The point is not that addition is itself intelligence. It is that sufficiently broad correct behavior can force a compact system to embody the general rule that explains the behavior, because exhaustive case enumeration exceeds the available resources.

A complexity-theoretic reconstruction

Let D_n be a domain of inputs of size n, $f_n : D_n \to R_n$ the target behavior, and s the description length of the responding system. A literal lookup implementation may require on the order of

$$|D_n| \log |R_n|$$

bits, whereas an algorithm exploiting shared structure may have description length $K(f_n) + O(\log n)$, where $K(f_n)$ informally denotes the length of a compact program for the family. If $|D_n|$ grows exponentially while feasible s grows only polynomially, exhaustive lookup is ruled out on resource grounds.

This does not prove that the compact program uses the same concepts or internal representation as a person. Many algorithms can compute the same function. It establishes the weaker and important conclusion that performance over a large, systematically variable domain cannot be dismissed as a finite bag of memorized answers.

The connection to statistical learning is immediate. Suppose a system matches f only on a benchmark sample $S \subset D$. Then empirical agreement

$$\widehat{R}_S(g) = \frac{1}{|S|} \sum_{x \in S} \mathbf{1}\{g(x) \neq f(x)\} = 0$$

does not imply small error on new inputs. Evidence for competence requires generalization to adaptively selected, structurally novel cases, not merely reproduction of a known test set.

Consequences for behavioral tests

Levesque's defense of behavioral evidence is conditional:

- the interaction must range over enough cases to defeat scripts and lookup tables;
- questions should be adaptive, cooperative, and open-ended rather than confined to a narrow contest;
- the system should handle recombinations, explanations, corrections, and novel circumstances; and
- resource bounds—memory, program size, training information, and computation—belong in the analysis.

Restricted chat programs can exploit cues and evasions without demonstrating the target ability. Conversely, if a system robustly responds across an effectively unbounded domain using feasible resources, the hypothesis that it has learned a general procedure becomes far more plausible.

Limits of the conclusion

The argument supports an inference from rich behavior to competence, not a deduction from behavior to subjective experience. Compactness alone is also insufficient: a short program may exploit a shortcut unrelated to the intended reasoning, and its training data can conceal a vast amount of memorized information. Modern evaluation must therefore combine held-out behavior with interventions, distribution shifts, mechanistic evidence, and careful accounting of training exposure.

1.4 Synthesis: Behavior, Representation, and General Competence

The three papers answer different levels of one problem:

1. Turing asks what public evidence could make machine intelligence an empirical question.
2. Levesque asks when behavioral success is too broad to be explained by rote enumeration and therefore supports an inference to general competence.
3. LeCun, Bengio, and Hinton describe how layered learning systems can acquire reusable representations that make such generalization possible.

Together they suggest an evaluation triangle:

$$\boxed{\text{behavioral breadth}} + \boxed{\text{resource-feasible mechanism}} + \boxed{\text{generalization beyond observed cases}}.$$

None of the vertices is sufficient alone. A convincing intelligent system must perform, do so through a plausible reusable mechanism, and remain competent on novel variations. This standard is stronger than benchmark accuracy while remaining empirically investigable; it is also agnostic about whether the machine duplicates human internal representations exactly.

2 First Networks

2.1 The Perceptron (Link)

The paper's object of study

Rosenblatt proposes the perceptron as a probabilistic model of how a biological system might store associations, recognize patterns, and generalize without a fully prewired symbolic program. The paper is broader than the single linear classifier now called a perceptron. Its central architecture has three kinds of units:

S-units receive a sensory stimulus, modeled most concretely as a retinal pattern;

A-units form an association layer reached through largely random connections; and

R-units produce competing responses and receive modifiable connections from the association system.

The random wiring makes the paper a theory of a population of possible networks. Rosenblatt consequently asks for probabilities of recognition and generalization, rather than claiming that one hand-designed circuit must work for every input.

Information is not stored as a literal copy of the stimulus. Learning changes the strength or effectiveness of connections between active association units and response units. A later stimulus produces the learned response insofar as it activates a sufficiently overlapping association pattern. This supplies the paper's basic account of generalization: similar sensory patterns tend to have correlated internal activity.

Modern linear-classifier abstraction

The descendant most often meant by *perceptron* maps $x \in \mathbb{R}^d$ to

$$\widehat{y} = \operatorname{sign}(w^\top x + b), \qquad y \in \{-1, +1\}.$$

Upon a mistake on (x_t, y_t), the online learning rule is

$$w_{t+1} = w_t + \eta y_t x_t, \qquad b_{t+1} = b_t + \eta y_t.$$

If the prediction is correct, the parameters are unchanged. Augmenting x by a constant coordinate absorbs b into w, so it suffices to analyze the homogeneous rule with $\eta = 1$.

This abstraction captures reinforcement of connections supporting the desired response, but it should not be retroactively identified with every detail of Rosenblatt's stochastic S–A–R model.

Proposition 2.1 (Perceptron convergence theorem)**.** *Suppose $\|x_t\| \le R$ and there is a unit vector u such that $y_t u^\top x_t \ge \gamma > 0$ for every training example. Starting from $w_0 = 0$, the perceptron makes at most*

$$M \le \left(\frac{R}{\gamma}\right)^2$$

mistakes.

Proof. After the Mth mistake, repeated use of the margin assumption gives

$$u^\top w_M = \sum_{t \in \mathcal{M}} y_t u^\top x_t \ge M\gamma.$$

On a mistaken example, $y_t w_t^\top x_t \le 0$, so

$$\|w_{t+1}\|^2 = \|w_t\|^2 + 2y_t w_t^\top x_t + \|x_t\|^2 \le \|w_t\|^2 + R^2.$$

Hence $\|w_M\| \le R\sqrt{M}$. Cauchy–Schwarz now yields $M\gamma \le u^\top w_M \le R\sqrt{M}$, giving the result. □

This later theorem clarifies both the achievement and limitation of the rule. It guarantees finite convergence only under linear separability with positive margin. It neither guarantees calibrated probabilities nor solves problems, such as XOR, whose classes cannot be separated by one hyperplane.

Probabilistic generalization in Rosenblatt's model

Let $a(x) \in \{0,1\}^m$ denote the active association units produced by a stimulus x. A simple response score can be written

$$s_r(x) = \sum_{j=1}^{m} w_{rj} a_j(x), \qquad \widehat{r}(x) = \arg\max_r s_r(x).$$

If training on x increases w_{rj} for active $a_j(x)$, then a new x' benefits from the same update in proportion to the overlap

$$\langle a(x), a(x') \rangle = \sum_{j=1}^{m} a_j(x) a_j(x').$$

Random S-to-A connections induce a random feature map. Recognition improves when stimuli judged similar by the environment activate overlapping features, while enough A-units are available to keep unrelated patterns distinguishable. This tension anticipates modern questions about random features, kernel similarity, capacity, and interference.

Scientific contribution and limitations

The paper's major conceptual moves are adaptive connection strengths, distributed internal activity, and statistical analysis of generalization. It argues that organized behavior can emerge from learning in a partly random network rather than from a complete symbolic wiring diagram.

Its biological vocabulary is heuristic, and the simplifying assumptions do not constitute a faithful model of cortical learning. The single-layer descendant has a sharp representational limitation; multilayer networks require a way to assign credit to hidden units. That missing credit-assignment mechanism is the problem attacked by the next paper.

2.2 Learning Representations by Back-Propagating Errors (Link)

From fixed features to learned hidden representations

Rumelhart, Hinton, and Williams present backpropagation as a procedure for learning the weights of a multilayer network. In a perceptron, the desired state of an output unit directly indicates how its incoming weights should change. A hidden unit has no target supplied by the data. Backpropagation solves this *credit-assignment problem* by computing how a small change in each hidden activation would affect the output error.

For unit j, let

$$x_j = \sum_i y_i w_{ji}, \qquad y_j = \sigma(x_j),$$

where the paper uses a smooth, monotonically increasing activation such as the logistic function $\sigma(x) = (1 + e^{-x})^{-1}$. For output targets d_j, its squared-error objective is

$$E = \frac{1}{2} \sum_{j \in \text{out}} (d_j - y_j)^2.$$

Deriving the error recursion

Define the local error signal

$$\delta_j = -\frac{\partial E}{\partial x_j}.$$

For an output unit,

$$\delta_j = (d_j - y_j)\sigma'(x_j).$$

For a hidden unit, the chain rule gives

$$\delta_j = \sigma'(x_j) \sum_{k:j \to k} w_{kj}\delta_k.$$

Finally,

$$-\frac{\partial E}{\partial w_{ji}} = \delta_j y_i, \qquad \Delta w_{ji} = \eta \delta_j y_i.$$

The two computational passes are therefore:

1. propagate activations forward to compute the output and loss;
2. propagate error derivatives backward, then form each weight gradient from a postsynaptic error signal times a presynaptic activation.

The same parameter may receive contributions from several cases or several uses in a recurrent computation; those contributions must be summed before or during the update.

Remark 2.1. *Backpropagation is reverse-mode automatic differentiation specialized to a layered graph. The paper helped make the method central to representation learning, but neither the multivariable chain rule nor reverse accumulation was historically unique to this article.*

Momentum and the geometry of optimization

The paper discusses accumulating gradients and using momentum. A modern form is

$$v_{t+1} = \mu v_t - \eta \nabla E(\theta_t), \qquad \theta_{t+1} = \theta_t + v_{t+1}.$$

Momentum averages aligned gradients across iterations, accelerates movement along persistent directions, and damps oscillation in steep directions. It does not remove nonconvexity. Symmetries such as permuting hidden units create many equivalent parameterizations, and saturation can make derivatives extremely small.

What the experiments demonstrate

The examples are designed to show that hidden codes can capture latent structure not explicitly named in the targets. A network can learn symmetry properties, relational encodings, and the regularities in a small family-tree domain. Internal units develop patterns corresponding to useful factors, and these factors permit correct responses on combinations withheld from training.

The important claim is thus stronger than function fitting: gradient descent can construct an internal representation that exposes task structure. Yet the representations are not guaranteed to be unique, human-readable, or causal. The experiments are small demonstrations rather than general statistical guarantees.

Technical boundaries

Differentiability permits credit assignment but does not guarantee successful learning. Deep sigmoid networks multiply many derivatives, producing vanishing gradients; recurrent networks repeat this multiplication through time. The objective can have saddles, plateaus, and local minima. Generalization further depends on the data, architecture, optimization path, and effective capacity. Backpropagation answers *how to compute the gradient*; it does not by itself answer whether the chosen model, loss, or data identify the desired rule.

2.3 Backpropagation Applied to Handwritten Zip Code Recognition (Link)

Problem and methodological shift

LeCun and coauthors apply gradient-based multilayer learning to handwritten ZIP code recognition. The work is significant because it integrates preprocessing, architectural constraints, and end-to-end parameter learning on a realistic vision problem. Rather than extract a large set of handcrafted geometric features and train a separate classifier, the network learns intermediate features that minimize recognition error.

Digit images are normalized to a common input grid. The output layer contains one unit per digit class. Training uses backpropagation, with choices such as weight sharing, local connectivity, stochastic updates, and momentum making the method computationally practical.

Local receptive fields and shared weights

A fully connected layer from an $H \times W$ image to m hidden units requires mHW weights and ignores spatial structure. If each hidden unit sees a $k \times k$ neighborhood, the unshared parameter count becomes mk^2. If one kernel is reused at all locations, the count drops to k^2 per feature map:

$$h_c(u, v) = \sigma\left(b_c + \sum_{a,b} K_c(a, b)x(u + a, v + b)\right).$$

The constraints encode two inductive biases: nearby pixels interact strongly, and the same visual feature may occur at different locations. They also cause all spatial uses of a kernel to contribute to one gradient,

$$\frac{\partial E}{\partial K_c(a, b)} = \sum_{u,v} \frac{\partial E}{\partial z_c(u, v)} x(u + a, v + b).$$

This is the learning analogue of statistical pooling across translated occurrences of a feature.

The architecture in the paper belongs to the lineage that became convolutional neural networks, although terminology and layer designs differ from current standard CNNs. Subsampling and staged local processing progressively enlarge the effective receptive field while retaining spatial organization.

Why architectural bias matters

Weight sharing restricts the hypothesis class. This can introduce bias if a feature truly has different meanings at different positions, but it reduces variance and sample requirements when translation is a genuine symmetry. The comparison is not simply "neural network versus classical classifier"; it is between models with different structural assumptions and different ways of using the image geometry.

For a stack of L stride-one convolutions with odd kernel widths $k_1, \ldots, k_L$, the receptive-field width is

$$r_L = 1 + \sum_{\ell=1}^{L} (k_\ell - 1).$$

With strides s_ℓ, the recursion becomes

$$r_\ell = r_{\ell-1} + (k_\ell - 1) \prod_{j<\ell} s_j.$$

Thus depth composes local detectors into features depending on a larger portion of the digit.

Experimental lesson and caveats

The reported postal-digit experiments show that a network trained from examples can achieve low error and learn feature detectors suited to the task. Rejection of uncertain cases can trade coverage for accuracy, which is operationally important in automated mail sorting.

The study predates present conventions for dataset scale, uncertainty quantification, calibration, and repeated hyperparameter evaluation. Image normalization supplies substantial prior knowledge, and performance on one postal distribution does not imply robustness to new writers, scanners, or countries. Its durable result is architectural: local connectivity and shared parameters make end-to-end visual learning statistically and computationally effective.

2.4 Long Short-Term Memory (Link)

The long-term dependency problem

Hochreiter and Schmidhuber study recurrent networks trained by gradient methods. For a simple recurrence $h_t = \phi(W h_{t-1} + U x_t)$, an error signal propagated from time T to time t contains

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \operatorname{diag}(\phi'(a_k)) W.$$

Repeated multiplication usually makes the norm decay exponentially or grow without bound. Relevant events separated by hundreds of steps then receive either no useful learning signal or an unstable one. The paper's goal is not merely to retain activations; it is to preserve *error flow* so gradient descent can learn what should be retained.

Constant error carousel

Consider an internal state c_t with a linear self-connection of weight one:

$$c_t = c_{t-1} + u_t.$$

Then, along the self-connected path,

$$\frac{\partial c_T}{\partial c_t} = 1,$$

regardless of $T - t$. The paper calls this fixed-weight loop a *constant error carousel* (CEC). It avoids both exponential decay and growth along the protected path.

A bare accumulator is vulnerable: irrelevant inputs can overwrite memory, and its state can perturb the rest of the network at inappropriate times. LSTM adds multiplicative gates. In simplified notation for the original architecture,

$$i_t = \sigma(W_{ix}x_t + W_{ih}h_{t-1} + b_i), \qquad o_t = \sigma(W_{ox}x_t + W_{oh}h_{t-1} + b_o),$$
$$\widetilde{c}_t = g(W_{cx}x_t + W_{ch}h_{t-1} + b_c), \qquad c_t = c_{t-1} + i_t \odot \widetilde{c}_t,$$
$$h_t = o_t \odot h(c_t).$$

The input gate protects stored state from irrelevant perturbations; the output gate protects downstream units until the memory is needed. Multiple cells may share gates as a memory-cell block.

Remark 2.2 (Original versus modern LSTM)**.** *The 1997 construction has a fixed unit self-connection and does not contain the learned forget gate now standard in*

$$c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t.$$

The forget gate was introduced later. When $f_t = 1$, the modern equation reduces to the original constant-error path; when $f_t < 1$, it learns controlled decay.

Gradient preservation and its qualifications

In a modern gated cell, the direct state-to-state derivative is approximately

$$\frac{\partial c_T}{\partial c_t} = \prod_{k=t+1}^{T} \operatorname{diag}(f_k),$$

apart from additional recurrent dependencies through the gates. Values of f_k near one preserve gradients; values near zero intentionally erase them. LSTM therefore creates a learnable path with favorable derivative geometry, not an unconditional theorem that every gradient in the entire network remains constant. Gradients outside the CEC may still vanish or explode, and gates may saturate.

The paper truncates selected error paths where doing so does not interfere with the protected carousel. Its update cost is local in space and time and scales linearly with the number of weights per time step, making the method more practical than nonlocal long-lag algorithms considered at the time.

Experiments and interpretation

Synthetic tasks test whether a network can remember markers, reproduce sequences, or combine values across long intervals filled with distractors. LSTM learns dependencies spanning more than one thousand time steps in settings where then-standard recurrent algorithms fail or learn prohibitively slowly. These experiments isolate temporal credit assignment; they are not claims of general language understanding.

The lasting innovation is an architectural optimization principle: introduce a state path whose Jacobian is well conditioned, then learn gates controlling information access. Residual connections, highway networks, and gated state space mechanisms reuse closely related reasoning.

2.5 A Deep Probabilistic Language Model (Link)

The statistical problem

Bengio, Ducharme, Vincent, and Jauvin address the curse of dimensionality in language modeling. By the chain rule,

$$p(w_1, \dots, w_T) = \prod_{t=1}^{T} p(w_t \mid w_1, \dots, w_{t-1}).$$

An n-gram model approximates the conditioning history by the previous $n-1$ words. With vocabulary size V, there are V^{n-1} possible contexts, most of which are absent even from a large corpus. Count-based smoothing shares mass across shorter contexts, but it does not directly encode that words such as *cat* and *dog* can occur in similar contexts.

The paper proposes learning simultaneously:

1. a distributed feature vector for every vocabulary item; and
2. a neural conditional probability model acting on those vectors.

Each observed sentence can then improve predictions for exponentially many nearby word combinations in representation space.

Model architecture

Let $C \in \mathbb{R}^{m \times V}$ be an embedding matrix and let $C(w)$ denote the column for word w. For a fixed context of length $n-1$, concatenate the embeddings:

$$x_t = \left[C(w_{t-n+1})^\top, \dots, C(w_{t-1})^\top\right]^\top.$$

A hidden layer and output logits can be written

$$h_t = \tanh(d + Hx_t), \qquad z_t = b + Wx_t + Uh_t,$$

where the direct connection Wx_t is optional. The normalized conditional probability is

$$p_\theta(w_t = j \mid w_{t-n+1:t-1}) = \frac{e^{z_{t,j}}}{\sum_{k=1}^{V} e^{z_{t,k}}}.$$

All components—the table C and the parameters of the probability function—are optimized jointly.

The penalized log-likelihood is

$$L(\theta) = \sum_t \log p_\theta(w_t \mid w_{t-n+1:t-1}) - R(\theta),$$

with weight decay as one possible regularizer. Stochastic gradient ascent uses each observed context–target pair to update both the predictor and the embeddings that appeared in its context.

Softmax and embedding gradients

For a single target y, let $p = \mathrm{softmax}(z)$. Then

$$\frac{\partial[-\log p_y]}{\partial z_j} = p_j - \mathbf{1}\{j = y\}.$$

Hence

$$\nabla_U \ell = (p - e_y)h^\top, \qquad \nabla_h \ell = U^\top(p - e_y).$$

Backpropagating through h and the concatenation gives a gradient for each context embedding. Words occurring in contexts that demand similar predictions receive similar update directions, providing the mechanism by which semantic and syntactic proximity can emerge without feature labels.

The normalization costs $O(V)$ per example because every vocabulary logit contributes to the denominator. The paper devotes substantial attention to parallelization and computational efficiency. This bottleneck later motivated hierarchical softmax, sampled objectives, noise-contrastive estimation, and other approximate normalization methods.

Evaluation by perplexity

For a corpus of T predicted tokens, cross-entropy and perplexity are

$$\widehat{H} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(w_t \mid c_t), \qquad \mathrm{PPL} = e^{\widehat{H}}.$$

Perplexity is the geometric mean inverse probability assigned to the observed next word. Lower perplexity means better probabilistic prediction under a fixed tokenization and evaluation protocol; values are not directly comparable when the vocabulary or tokenization differs.

The experiments on the Brown and Associated Press corpora show improved perplexity relative to conventional smoothed n-gram baselines, with further gains from combining neural and trigram predictions. The complementarity suggests that distributed similarity and exact local counts capture different regularities.

Conceptual legacy and limitations

The paper establishes the template

$$\text{discrete symbol} \longrightarrow \text{learned vector} \longrightarrow \text{differentiable probability model},$$

which underlies modern neural language models. Its fixed context window cannot represent arbitrary long-range dependencies, its full softmax is expensive, and a single vector per word conflates context-dependent meanings. Later recurrent and transformer models replace the fixed-window predictor with a contextual sequence model, while retaining joint representation learning and maximum-likelihood training.

2.6 Synthesis: The Emergence of Representation Learning

These five papers trace a coherent sequence of technical problems and answers:

1. Rosenblatt shows how adaptive connections and distributed activity can associate stimuli with responses and generalize through overlap.
2. Rumelhart, Hinton, and Williams provide the gradient mechanism needed to learn hidden features rather than fix them in advance.
3. LeCun and coauthors encode spatial locality and translation structure, making multilayer learning practical for images.
4. Hochreiter and Schmidhuber redesign recurrent state so gradients can traverse long temporal gaps.
5. Bengio and coauthors learn continuous representations of discrete words jointly with a normalized probabilistic model.

Across the group, successful learning depends on both an objective and an architecture whose parameter sharing and gradient geometry reflect the problem. Backpropagation is the common computational engine, but locality, recurrence, gating, and embeddings determine what can be represented and which statistical regularities are shared.

3 Words as Numbers

3.1 Efficient Estimation of Word Representations in Vector Space (Link)

Problem and design principle

Mikolov, Chen, Corrado, and Dean seek word representations that can be trained on billions of tokens and vocabularies containing millions of words. Earlier neural language models learn useful vectors, but their nonlinear hidden layers and full-vocabulary output normalization are costly. The paper's design principle is therefore to sacrifice some modeling capacity per example in exchange for processing vastly more data.

Let the vocabulary be $\mathcal{V}$ with $|\mathcal{V}| = V$, and let $W \in \mathbb{R}^{V \times d}$ contain an input vector v_w for each word. Training constructs prediction tasks from a corpus $w_1, \ldots, w_T$. No semantic labels are supplied: distributional structure is learned from which words occur near one another.

The authors summarize training complexity as

$$O(ETQ),$$

where E is the number of epochs, T is the number of training tokens, and Q is the architecture-dependent work per token. Removing a dense nonlinear hidden layer eliminates terms such as dH or H^2 that dominate earlier feed-forward and recurrent neural language models.

Continuous bag of words

The continuous bag-of-words model (CBOW) predicts a center word from the words around it. For a context index set C_t, its projection is the average

$$h_t = \frac{1}{|C_t|} \sum_{j \in C_t} v_{w_j}.$$

If $u_w \in \mathbb{R}^d$ denotes the output vector for word w, a full-softmax version would use

$$p(w_t = w \mid C_t) = \frac{\exp(u_w^\top h_t)}{\sum_{w' \in \mathcal{V}} \exp(u_{w'}^\top h_t)}$$

and minimize

$$\mathcal{L}_{\text{CBOW}} = -\sum_t \log p(w_t \mid C_t).$$

Because the context embeddings are averaged, their order is discarded. The same embedding table is used at every context position, sharing statistical strength across locations. The paper's strongest configuration uses words on both sides of the target; this is a representation-learning objective rather than a causal left-to-right language model.

For a context of N words and hierarchical output of depth approximately $\log_2 V$, the paper gives per-example complexity

$$Q_{\text{CBOW}} = Nd + d \log_2 V.$$

CBOW is especially efficient because all context words are combined before one output prediction.

Continuous skip-gram

Skip-gram reverses the conditional direction: a center word predicts nearby words. Given a maximum window radius C, a basic objective is

$$\mathcal{L}_{\text{SG}} = -\sum_{t=1}^{T} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \log p(w_{t+j} \mid w_t).$$

The paper samples the actual radius R uniformly from $\{1, \dots, C\}$ for each center, so distant words are included less often than close words. With a hierarchical output, its reported complexity is proportional to

$$Q_{\text{SG}} = C\Big(d + d \log_2 V\Big).$$

Skip-gram performs several predictions per center word and is consequently slower than CBOW, but in the paper it captures semantic relations particularly well. CBOW's averaging makes it statistically smoother and gives strong syntactic performance at lower cost.

The two parameterizations can be viewed as low-rank conditional models:

$$\text{score}(w, c) = u_w^\top v_c.$$

The input and output vectors play different roles. A released "word vector" is usually an input vector v_w, although later practice also uses or combines both tables. Treating them as automatically interchangeable obscures the asymmetry of the prediction objective.

Hierarchical softmax

A conventional softmax requires V scores per prediction. The paper instead places words at leaves of a binary Huffman tree. Let $n_1(w), \dots, n_{L(w)-1}(w)$ be the internal nodes on the path to word w, and let $b_j(w) \in \{-1, +1\}$ encode whether the path turns left or right. Then

$$p(w \mid h) = \prod_{j=1}^{L(w)-1} \sigma\Big(b_j(w) u_{n_j(w)}^\top h\Big).$$

Every internal decision is a normalized Bernoulli probability, so summing the leaf probabilities gives one. Computing a word probability touches only the nodes along its path, reducing output work from $O(Vd)$ to $O(d \log V)$. A Huffman tree assigns shorter paths to frequent words, lowering the expected path length under the empirical unigram distribution.

For one node, with $a = b, u^\top h$, the negative log-likelihood derivative is

$$\nabla_u[-\log \sigma(a)] = -b\{1 - \sigma(a)\}h.$$

Thus training moves the hidden representation and each path vector toward the correct binary decisions.

Remark 3.1 (Negative sampling is later work)**.** *The submitted paper uses hierarchical softmax for its large-vocabulary experiments. The now-famous negative-sampling objective was introduced in the follow-up paper* Distributed Representations of Words and Phrases and Their Compositionality*. It should not be attributed to this paper merely because both methods are implemented by the word2vec software.*

Why linear relations can appear

The learned geometry is evaluated through cosine similarity,

$$\cos(v, w) = \frac{v^\top w}{\|v\| \, \|w\|},$$

and vector-offset questions. Given a relation pair $a : b$ and a query word c, the familiar 3CosAdd rule selects

$$\widehat{d} = \arg \max_{d \notin \{a,b,c\}} \cos\Big(v_d, , v_b - v_a + v_c\Big).$$

If a relation produces approximately constant displacement, $v_b - v_a \approx r$ and $v_d - v_c \approx r$, the nearest-neighbor search recovers d. Morphological transformations can create particularly regular offsets because related word forms appear in systematically transformed contexts.

This phenomenon is empirical, not a theorem that semantic relations are intrinsically linear. The training loss constrains dot products used for prediction, and many coordinate systems represent the same scores. For example, for any invertible A,

$$v'_w = A v_w, \qquad u'_c = A^{-\top} u_c \quad \Longrightarrow \quad u'^\top_c v'_w = u_c^\top v_w.$$

The predictive model is therefore non-identifiable. General invertible changes need not preserve cosine geometry, whereas a common orthogonal transformation does preserve dot products, lengths, cosines, and analogy offsets. Claims about individual coordinates require additional identification assumptions.

Evaluation and empirical findings

The paper constructs a semantic–syntactic analogy set with five semantic and nine syntactic relation types, totaling 8,869 semantic and 10,675 syntactic questions. A prediction counts as correct only when the nearest word is the designated answer; plausible synonyms are marked incorrect, and multiword entities are excluded.

The principal findings are:

- increasing both corpus size and embedding dimension improves analogy accuracy, with diminishing returns if only one is increased;
- CBOW trains faster and is particularly effective on syntactic regularities;
- skip-gram is more expensive but substantially improves the semantic portion of the authors' benchmark;
- simple log-linear models trained on more data outperform more elaborate neural language models as sources of vectors; and
- skip-gram scores complement recurrent language-model scores on the Microsoft Sentence Completion Challenge, even though skip-gram alone is not the strongest sentence model.

These results support the paper's scale-versus-complexity thesis: a cheaper objective may produce better reusable representations by seeing more examples at higher dimension.

Statistical interpretation and limitations

Both objectives use nearby co-occurrence as a surrogate for meaning. They therefore learn regularities of the corpus, including its topical structure, frequency effects, stereotypes, and social biases. Proximity means distributional substitutability or association; it does not by itself establish reference, entailment, causal knowledge, or factual truth.

Several limitations follow from representing each word type by one fixed vector:

- polysemous words conflate multiple senses;
- unseen words have no learned vector, and internal morphology is ignored;
- bagged local windows discard order in CBOW;
- analogy accuracy is sensitive to corpus composition, vocabulary, nearest-neighbor rule, and benchmark construction; and
- high similarity can reflect raw frequency or shared topic rather than the relation intended by an evaluator.

The paper's lasting contribution is thus not the claim that all linguistic knowledge is vector arithmetic. It demonstrates that an aggressively simplified self-supervised prediction problem, paired with parameter sharing and efficient normalization, can turn very large unlabeled corpora into useful continuous representations. Contextual language models later replace one

vector per word with a representation depending on the entire occurrence, while preserving this principle of learning representations through prediction.

4 Going Deep

4.1 Deep Sparse Rectifier Neural Networks (Link)

Rectifiers as an optimization and representation choice

Glorot, Bordes, and Bengio study the rectified linear unit

$$\rho(z) = \max\{0, z\}, \qquad \rho'(z) = \mathbf{1}\{z > 0\}$$

(away from the nondifferentiable point $z = 0$). Their experiments show that deep supervised networks with rectifiers can match or exceed then-standard sigmoid networks without unsupervised layerwise pretraining. The contribution is not merely a different scalar nonlinearity: rectifiers change gradient propagation, activation sparsity, and the geometry of the represented function.

A sigmoid derivative satisfies $0 < \sigma'(z) \leq 1/4$ and becomes tiny when the unit saturates. An active rectifier has derivative one, so it transmits its incoming gradient without multiplicative shrinkage. An inactive rectifier has exactly zero activation and derivative, yielding sparse codes but also creating the possibility of a permanently inactive or "dead" unit.

Piecewise-linear geometry

For a fixed pattern of active units, let $D_\ell(x)$ be the diagonal matrix of binary activation indicators in layer ℓ. A rectifier network then reduces locally to an affine map with Jacobian

$$J_f(x) = W_L D_{L-1}(x) W_{L-1} \cdots D_1(x) W_1.$$

Input space is partitioned into polyhedral regions indexed by activation patterns, and the network is affine on each region. Depth can reuse and compose boundaries, producing many linear regions without requiring an explicit unit for every region.

Positive homogeneity,

$$\rho(az) = a\rho(z), \qquad a \geq 0,$$

also creates scale non-identifiability: multiplying the incoming weights of one unit by a and dividing its outgoing weights by a leaves the network function unchanged. Parameter norms must therefore be interpreted jointly across layers.

Sparsity and regularization

Real zeros arise from the activation itself. The paper additionally studies an L^1 penalty on activations,

$$\widehat{R}(\theta) + \lambda \sum_{i,\ell} \left\| h_i^{(\ell)} \right\|_1,$$

which encourages a small number of active features. Sparse representations can reduce interference and yield features that respond selectively, although sparsity is not synonymous with disentanglement or interpretability.

Across image and text benchmarks, rectifiers improve supervised optimization and often produce sparse hidden activity. The result is empirical rather than a global convergence guarantee. Initialization, input scale, learning rate, and the probability of dead units remain important; later variants such as leaky ReLU alter the zero-slope half-line to mitigate permanent inactivity.

4.2 Dropout: A Simple Way to Prevent Neural Networks from Overfitting (Link)

Stochastic subnetworks

Dropout independently retains each non-output unit with probability p. If h is a layer activation and $m_j \sim \text{Bernoulli}(p)$, training uses

$$\widetilde{h} = m \odot h.$$

Every mask selects a subnetwork, while all subnetworks share parameters. This discourages units from relying on fragile co-adaptations with particular peers. The training criterion is an expectation over masks,

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}_m \Big[\ell(f_{\theta,m}(x_i), y_i) \Big],$$

estimated with one or a few sampled masks per mini-batch.

In the paper's convention, activations are unscaled during training and outgoing weights are multiplied by p at test time. The now-common *inverted dropout* instead uses $\widetilde{h} = (m/p) \odot h$ during training and makes no test-time rescaling. Both conventions match the first moment.

Proposition 4.1 (Exact mean prediction for a linear next layer)**.** *If $z = W(m \odot h) + b$, then*

$$\mathbb{E}_m[z] = W(ph) + b.$$

Thus weight scaling exactly averages the preactivation of the exponentially many masked linear models.

Proof. Linearity of expectation and $\mathbb{E}[m_j] = p$ give $\mathbb{E}[W(m \odot h) + b] = W(\mathbb{E}[m] \odot h) + b = W(ph) + b$. □

After a nonlinearity, generally $\mathbb{E}[\phi(z)] \neq \phi(\mathbb{E}[z])$. The deterministic test network is consequently an approximation to ensemble averaging, not an exact average of an arbitrary deep network's predictions.

Noise and adaptive regularization

Under inverted dropout,

$$\mathbb{E}[\widetilde{h}_j] = h_j, \qquad \mathrm{Var}(\widetilde{h}_j) = \frac{1-p}{p} h_j^2.$$

The noise is multiplicative and its variance grows with activation magnitude. For generalized linear models, a second-order expansion connects dropout to a data-dependent quadratic penalty: directions with large feature magnitude or curvature are penalized more strongly. This differs from isotropic weight decay.

The paper reports gains across vision, speech, document classification, and computational biology, often combined with max-norm constraints, momentum, and large learning rates. Dropout usually slows optimization because the objective is noisier; its purpose is improved generalization. At inference, Monte Carlo dropout can retain stochastic masks to estimate predictive variation, but that Bayesian interpretation is later work and is not established by this paper.

4.3 ImageNet Classification with Deep Convolutional Neural Networks (Link)

What changed at ImageNet scale

Krizhevsky, Sutskever, and Hinton train a large convolutional network on roughly 1.2 million images in 1000 classes. The model has five convolutional and three fully connected learned layers, about 60 million parameters, and is split across two GPUs. It achieves a large improvement in the 2012 ImageNet competition, making the combination of deep CNNs, GPU computation, and large labeled data decisive for computer vision.

The classifier minimizes multinomial negative log-likelihood. Its notable components are:

- ReLUs, which train substantially faster than saturating tanh units;
- overlapping max pooling;
- data augmentation by random crops, horizontal reflections, and RGB intensity perturbations;
- dropout in the fully connected layers;
- SGD with momentum and weight decay; and
- local response normalization, an historically important component that is not essential in most later architectures.

Augmentation as an invariance prior

Let G be a distribution over label-preserving transformations such as crops and reflections. Augmented training minimizes

$$\widehat{R}_{\mathrm{aug}}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_{g\sim G}\ell(f_\theta(gx_i), y_i).$$

This is stochastic integration over a transformation orbit. It encourages, but does not mathematically guarantee, $f(gx) = f(x)$. Test-time averaging over crops similarly approximates

$$p(y \mid x) \approx \frac{1}{K}\sum_{k=1}^{K} p_\theta(y \mid g_k x).$$

The RGB perturbation based on principal components injects variation along dominant color directions, reducing dependence on illumination statistics.

Systems and scientific lessons

The paper's success cannot be assigned to a single novel layer. Convolutional parameter sharing makes image learning tractable; ReLUs improve gradient flow; augmentation and dropout control overfitting; GPUs make the experiment possible; and ImageNet supplies task diversity and scale. This co-design of model, optimization, data, and hardware became the template for modern deep learning.

Top-k ImageNet accuracy measures recognition under one curated distribution. It does not imply robustness to adversarial perturbations, corruptions, class imbalance, or deployment shift. The two-GPU grouping and local response normalization are contingent engineering choices, while learned hierarchical visual representations and large-scale accelerator training are the enduring ideas.

4.4 Adam: A Method for Stochastic Optimization (Link)

Algorithm

Adam combines momentum-like first moments with coordinatewise second moments. Given stochastic gradient $g_t = \nabla_\theta f_t(\theta_{t-1})$, it computes

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^{\odot 2},$$

where squares, square roots, and divisions involving v_t are elementwise. Starting from zero makes both moving averages biased toward zero. Adam corrects this transient:

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \widehat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \alpha\frac{\widehat{m}_t}{\sqrt{\widehat{v}_t}+\epsilon}.$$

If $g_1, g_2, \ldots$ have a stationary mean μ, then

$$\mathbb{E}[m_t] = (1-\beta_1^t)\mu,$$

which directly explains the first bias correction. The analogous calculation holds for the uncentered second moment. The denominator normalizes a coordinate by its recent root-mean-square gradient, while the numerator smooths its signed direction.

Scale behavior and effective steps

Ignoring ϵ, multiplying every gradient in coordinate j by a positive constant c multiplies $\widehat{m}_{t,j}$ by c and $\sqrt{\widehat{v}_{t,j}}$ by c, leaving the update unchanged. Adam is thus approximately invariant to diagonal rescaling of gradients. Its effective coordinatewise learning rate is

$$\alpha_{t,j}^{\text{eff}} = \frac{\alpha}{\sqrt{\widehat{v}_{t,j}} + \epsilon}.$$

Sparse or infrequent coordinates can receive relatively large steps, which is useful in embedding models.

The paper supplies an online convex-optimization regret analysis under bounded gradients, bounded iterates, decaying effective step sizes, and a relationship between the moment parameters. Its intended conclusion is sublinear regret, so average regret vanishes. This theory concerns a constrained convex setting; it is not a proof of convergence to global optima in deep nonconvex networks.

Subsequent convergence caveat

Later work exhibited convex examples on which the original Adam update fails to converge because the exponentially weighted second moment can decrease, causing an effective learning rate to grow in an unfavorable coordinate. AMSGrad addresses this by replacing v_t with the running maximum

$$\widetilde{v}_t = \max(\widetilde{v}_{t-1}, v_t).$$

This historical correction is essential when treating Adam theoretically: the paper's algorithm remains enormously influential empirically, but its original general convergence claim should not be read without the later qualifications. Also, coupled L^2 regularization is not equivalent to multiplicative weight decay under adaptive preconditioning; AdamW explicitly decouples the latter.

4.5 Batch Normalization (Link)

Training-time transformation

For a scalar preactivation x in a mini-batch $B = \{x_1, \dots, x_m\}$, batch normalization computes

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2,$$

$$\widehat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y_i = \gamma \widehat{x}_i + \beta.$$

The learned scale and shift preserve representational capacity: the layer can recover an appropriate mean and variance rather than being forced to output a standardized variable. In convolutional networks, statistics are usually shared over the mini-batch and spatial positions separately for each channel.

At inference, batch statistics are replaced by population estimates accumulated during training. Training and evaluation therefore implement different functions unless these estimates agree. Very small or nonrepresentative batches can make the stochastic statistics noisy or biased toward the batch composition.

Reparameterization and scale invariance

Ignoring ϵ, scaling all incoming weights of a normalized unit by $a > 0$ changes x_i to ax_i, μ_B to $a\mu_B$, and σ_B to $a\sigma_B$, leaving $\widehat{x}_i$ unchanged. Thus

$$\mathrm{BN}(aWx) = \mathrm{BN}(Wx).$$

Differentiating this invariance implies that the loss gradient is orthogonal to the weight vector:

$$W^\top \nabla_W L = 0$$

in the idealized setting. Batch normalization changes the parameter geometry and makes the represented function less sensitive to weight scale, often permitting larger learning rates.

The derivative couples examples within a mini-batch. If $g_i = \partial L/\partial y_i$, then

$$\frac{\partial L}{\partial x_i} = \frac{\gamma}{\sqrt{\sigma_B^2 + \epsilon}} \left[g_i - \overline{g} - widehatx_i \, overlineg\widehat{x} \right],$$

where bars denote batch averages. Centering both g and its component aligned with $\widehat{x}$ explains why BN is not merely a fixed preprocessing step.

Interpretation and limitations

Ioffe and Szegedy motivate BN as reducing "internal covariate shift," meaning changes in intermediate activation distributions during training. Later analyses show that this explanation is neither necessary nor sufficient; loss surface smoothing, scale invariance, and gradient conditioning better explain many benefits. The paper's empirical claims remain important even if the original causal story is incomplete.

BN also introduces batch dependence and stochastic regularization. It can be awkward for recurrent models, online prediction, privacy-sensitive batches, or distribution shifts that alter activation statistics. Layer normalization and group normalization remove dependence on other examples by normalizing over different axes.

4.6 Deep Residual Learning for Image Recognition (Link)

The degradation problem

He, Zhang, Ren, and Sun distinguish degradation from classical overfitting. As depth increases, a plain network can have higher *training* error even though the deeper hypothesis class could represent the shallower solution by making added layers identities. The difficulty is optimization: a stack of ordinary nonlinear layers does not readily discover an exact identity map.

A residual block parameterizes

$$y = F(x; W) + x.$$

If the desired mapping is $H(x)$, the nonlinear branch learns $F(x) = H(x) - x$. Setting the residual branch near zero immediately supplies an identity-like solution. When dimensions differ, the shortcut may use a linear projection $W_s x$ instead of x.

Forward and backward identity paths

For a sequence of ideal identity-shortcut blocks

$$x_{ell+1} = x_\ell + F_\ell(x_\ell),$$

unrolling gives

$$x_L = x_\ell + \sum_{k=\ell}^{L-1} F_k(x_k).$$

The Jacobian of one block is

$$\frac{\partial x_{\ell+1}}{\partial x_\ell} = I + J_{F_\ell}(x_\ell).$$

Consequently the gradient contains a direct identity contribution rather than being only a product of learned Jacobians. For any loss L,

$$\frac{\partial L}{\partial x_\ell} = \frac{\partial L}{\partial x_L}\left(I + \frac{\partial}{\partial x_\ell}\sum_{k=\ell}^{L-1} F_k(x_k)\right).$$

This does not guarantee perfect conditioning—the residual Jacobians still matter—but it gives information and gradients a short unobstructed route.

For small J_F, singular values of $I + J_F$ remain near one. Residual blocks can therefore be interpreted as incremental perturbations of the identity, connecting very deep networks to discretizations of dynamical systems:

$$x_{\ell+1} - x_\ell = F_\ell(x_\ell)$$

resembles a unit-step forward Euler update.

Architecture and evidence

The ImageNet models use stacks of 3×3 convolutions and, for greater depth, bottleneck blocks with 1×1, 3×3, and 1×1 convolutions. Identity shortcuts add no parameters or major computation. The paper trains networks up to 152 layers, shows that residual networks avoid the training-error degradation of comparable plain networks, and reports leading ImageNet and transfer results on detection and localization tasks.

Residual learning does not imply that deeper is always statistically better. Compute, width, normalization, optimization, and data still constrain useful depth. Nor does every residual branch literally learn a small perturbation. The durable contribution is a parameterization that makes identity mappings and direct signal paths structurally easy, enabling optimization at depths that were previously impractical.

4.7 Synthesis: Making Depth Trainable

These papers solve complementary bottlenecks. ReLUs preserve derivatives on active paths; convolution and augmentation encode image symmetries; dropout regularizes large co-adapted networks; Adam adapts steps to gradient history; batch normalization improves parameter geometry; and residual shortcuts create identity routes through depth. Their combined lesson is that expressivity alone does not explain deep learning. Successful scaling requires control of gradient flow, stochastic regularization, inductive bias, numerical conditioning, data, and hardware.

5 From Translation to Transformers

5.1 Sequence to Sequence Learning with Neural Networks (Link)

Conditional sequence modeling

Sutskever, Vinyals, and Le formulate translation as learning a conditional distribution over a variable-length target sequence $y_{1:M}$ given a variable-length source sequence $x_{1:N}$:

$$p_\theta(y_{1:M} \mid x_{1:N}) = \prod_{t=1}^{M} p_\theta(y_t \mid y_{<t}, x_{1:N}).$$

An encoder LSTM reads the source and compresses it into its final state c. A decoder LSTM initializes from c and produces a distribution for each target token until an end-of-sentence symbol is emitted. Training maximizes

$$\mathcal{L}(\theta) = \sum_{(x,y)\in\mathcal{D}} \sum_{t=1}^{|y|} \log p_\theta(y_t \mid y_{<t}, c(x)).$$

During training the decoder receives the observed previous token; at inference it must condition on its own generated history. This train–test discrepancy is now called exposure bias.

The model is notable for making few assumptions about alignment or equal input and output lengths. The same differentiable objective learns lexical, syntactic, and ordering regularities end to end.

The fixed-vector bottleneck and source reversal

All source information must pass through the fixed-dimensional state c. For long or information-dense sentences this creates a compression and optimization bottleneck. The authors find that reversing the source order while leaving the target order unchanged materially improves performance. If source and target sentences have roughly corresponding left-to-right structure, reversal shortens the minimal computational path from early target tokens to their related source tokens. It does not make the languages linguistically reversed; it changes the topology through which gradients and information flow.

The encoder and decoder use separate deep LSTMs. LSTM gates mitigate vanishing gradients, gradient norms are clipped to control explosions, and sentence pairs of similar length are batched together for computational efficiency.

Decoding by beam search

The maximum a posteriori translation is

$$y^* = \arg\max_y \log p_\theta(y \mid x) = \arg\max_y \sum_t \log p_\theta(y_t \mid y_{<t}, x),$$

but exhaustive search is exponential in sequence length. Beam search retains only the B highest-scoring partial sequences after each expansion. With $B = 1$ it is greedy decoding; as B grows it explores more hypotheses but is still not guaranteed to find the global maximizer unless the beam is exhaustive.

Because log probabilities are nonpositive, raw sequence scores favor short hypotheses; later systems commonly use length normalization or explicit length penalties. These are decoding choices, not changes to the learned conditional distribution.

Evidence and limitations

The deep LSTM substantially improves English-to-French translation and learns sentence representations in which similar meanings cluster, while an ensemble and reranking setup approaches strong phrase-based systems. The experiments establish that one neural architecture can map between sequences without a hand-engineered alignment pipeline.

The single-vector interface remains the central weakness. It asks $c(x)$ to be sufficient for every target decision:

$$p(y_t \mid y_{<t}, x) = p(y_t \mid y_{<t}, c(x)).$$

This conditional-sufficiency approximation becomes increasingly demanding as N grows. The next paper replaces the one-vector interface with differentiable access to every encoder state.

5.2 Neural Machine Translation by Jointly Learning to Align and Translate (Link)

Soft alignment

Bahdanau, Cho, and Bengio let the decoder construct a different source summary for every target position. A bidirectional recurrent encoder produces annotations

$$h_j = [\overrightarrow{h}_j; \overleftarrow{h}_j],$$

which describe source position j using both left and right context. At target step i, an alignment network assigns energy

$$e_{ij} = v_a^\top \tanh(W_a s_{i-1} + U_a h_j),$$

then normalizes over source positions:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{N} \exp(e_{ik})}, \qquad c_i = \sum_{j=1}^{N} \alpha_{ij} h_j.$$

The decoder updates s_i and predicts y_i using s_{i-1}, the preceding target token, and c_i. This score is now called *additive attention*.

Proposition 5.1 (Context vectors lie in a convex hull)**.** *For each target step i, c_i belongs to the convex hull of $\{h_1, \ldots, h_N\}$.*

Proof. Softmax gives $\alpha_{ij} > 0$ and $\sum_j \alpha_{ij} = 1$, so c_i is a convex combination of the annotations. □

The context is therefore a smooth retrieval from source memory. Gradients flow through both the selected annotation and the alignment weights, so no supervised word alignment is required.

Why attention removes the bottleneck

In the fixed-vector model, every source-to-target dependency passes through $c(x)$. With attention, target position i has a short path to every h_j. The mechanism does not eliminate compression entirely—each h_j is finite dimensional and c_i is still an average—but it distributes memory across positions and makes relevant information directly addressable.

The weights α_{ij} often resemble linguistic alignments and can be visualized. They should not automatically be interpreted as a unique causal explanation: different parameterizations may yield similar predictions with different attention maps, and a diffuse convex combination may encode useful information without a one-to-one alignment.

Empirical and conceptual contribution

The attention model improves translation especially on long sentences, where the fixed-vector encoder degrades. It jointly learns translation and alignment using the translation likelihood alone. More generally, it introduces a content-addressable interface between neural modules:

$$\text{query } s_{i-1} \quad \longrightarrow \quad \text{scores } e_{ij} \quad \longrightarrow \quad \text{weighted memory read } c_i.$$

The model remains recurrent in both encoder and decoder, so states cannot be computed fully in parallel. The Transformer later retains differentiated memory access while removing recurrence.

5.3 Neural Machine Translation of Rare Words with Subword Units (Link)

The open-vocabulary problem

Word-level neural translation systems typically restrict output vocabularies to tens of thousands of types. Unseen names, compounds, inflections, and transliterations collapse to an unknown symbol or require an external dictionary. Sennrich, Haddow, and Birch instead represent a sentence as a sequence of subword units. Frequent words remain intact, while rare words are composed from reusable pieces.

The statistical motivation is a bias–variance tradeoff. Whole words preserve short sequences but give sparse estimates for rare types. Characters eliminate out-of-vocabulary words but lengthen sequences and make the model discover long-range composition. Data-driven subwords interpolate between these extremes.

Byte-pair encoding as vocabulary induction

The paper adapts byte-pair encoding (BPE), originally a compression algorithm:

1. initialize each word as a sequence of characters plus an end-of-word marker;
2. count adjacent symbol pairs in the training vocabulary, weighted by word frequency;
3. merge the most frequent pair into one new symbol; and
4. repeat for a prescribed number of merges.

The learned ordered merge list defines a deterministic tokenizer. If (a, b) is merged before (ab, c), a later word containing the sequence abc can inherit the hierarchical construction. The number of merges controls vocabulary size and average sequence length.

For multilingual use, learning BPE jointly over the source and target encourages shared spellings of names and cognates to receive shared units. Separate BPE models allow language-specific segmentation. Character coverage makes the system effectively open-vocabulary when every required base character is available.

Probabilistic consequences

If a word w is segmented into subwords $z_{1:k}$, the decoder assigns

$$p(w \mid \text{history}) = \prod_{r=1}^{k} p(z_r \mid z_{<r}, \text{history}),$$

with the caveat that the history evolves as pieces are generated. A rare word can receive probability through well-estimated components, but it now requires several correct autoregressive decisions. Segmentation therefore changes the sample space, sequence length, and calibration of token-level likelihoods.

The paper compares character n-gram and BPE segmentations and reports improved English–German and English–Russian translation, particularly for names, compounds, and cognates, without an external rare-word backoff system.

Subwords do not solve morphology or meaning in general. Greedy frequency merges optimize neither translation likelihood nor linguistic segmentation, the same surface piece may have different functions across contexts, and token boundaries can create brittle behavior. The enduring contribution is to make vocabulary construction part of the modeling pipeline rather than accept word types as atomic.

5.4 Attention Is All You Need (Link)

Scaled dot-product attention

Vaswani and coauthors replace recurrent sequence computation with attention. For queries $Q \in \mathbb{R}^{n_q \times d_k}$, keys $K \in \mathbb{R}^{n_k \times d_k}$, and values $V \in \mathbb{R}^{n_k \times d_v}$,

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V,$$

where softmax is rowwise and M may mask prohibited query–key pairs. Each output is a convex combination of value vectors, but the combination weights depend on content through query–key similarity.

If query and key coordinates are independent with mean zero and variance one, then

$$\text{Var}(q^\top k) = d_k.$$

Dividing by $\sqrt{d_k}$ keeps logits at order-one scale, reducing softmax saturation and small gradients as the head dimension grows.

Multi-head attention

One attention average may combine incompatible relations. The Transformer uses H learned projections:

$$\text{head}_h = \text{Attn}(QW_h^Q, KW_h^K, VW_h^V),$$

$$\mathrm{MHA}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H) W^O.$$

With total model width d_{model} split across heads, the aggregate projection cost remains comparable to one full-width attention layer. Heads can specialize to different positions or representation subspaces, although specialization is learned rather than guaranteed.

The encoder uses self-attention with $Q = K = V$ from the preceding layer. The decoder uses masked self-attention so position t cannot access future target tokens, followed by cross-attention whose queries come from the decoder and whose keys and values come from the encoder.

Permutation symmetry and positional encoding

Without positional information, self-attention is permutation equivariant. For a permutation matrix P and no mask,

$$\mathrm{Attn}(PQ, PK, PV) = P\,\mathrm{Attn}(Q, K, V).$$

Indeed, the score matrix becomes $P(QK^\top)P^\top$, rowwise softmax commutes with simultaneous row/column permutation, and multiplication by PV gives the result. Attention alone therefore represents a set operation rather than sequence order.

The paper adds sinusoidal encodings to token embeddings:

$$\mathrm{PE}(p, 2i) = \sin\left(p/10000^{2i/d}\right), \qquad \mathrm{PE}(p, 2i+1) = \cos\left(p/10000^{2i/d}\right).$$

For a fixed offset r, angle-addition identities express $\mathrm{PE}(p + r)$ as a blockwise linear transformation of $\mathrm{PE}(p)$, making relative offsets accessible. The paper reports similar results for learned positional embeddings; sinusoids offer a possible route to extrapolation beyond trained lengths, not a guarantee of it.

Encoder and decoder blocks

Each encoder layer combines multi-head self-attention with a positionwise feed-forward network

$$\mathrm{FFN}(x) = W_2 \max(0, W_1 x + b_1) + b_2.$$

Residual connections and layer normalization surround each sublayer. In the original paper this is a post-normalization form,

$$\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x));$$

the pre-normalization form common in later large language models is a subsequent architectural convention.

Training uses Adam, dropout, label smoothing, and a learning-rate schedule with linear warmup followed by inverse-square-root decay:

$$\eta_t = d_{\text{model}}^{-1/2} \min(t^{-1/2}, t, w^{-3/2}),$$

where w is the number of warmup steps.

Complexity and path length

For sequence length n and width d, a dense self-attention layer costs $O(n^2 d)$ and stores an $n \times n$ attention matrix; a recurrent layer costs roughly $O(nd^2)$. Self-attention is cheaper when $n < d$ and, crucially, all positions can be computed in parallel during training. The maximum path length between any two positions is $O(1)$ per self-attention layer rather than $O(n)$ through recurrence, facilitating long-range interactions.

The quadratic dependence on n becomes the Transformer's principal long-context bottleneck. Autoregressive decoding also remains sequential across generated tokens even though training with known targets is parallel across positions.

Results, scope, and legacy

The Transformer reaches leading WMT English–German and English–French results with substantially less training cost than prior recurrent ensembles and also transfers to constituency parsing. Its decisive contribution is an architecture whose core sequence interaction is content-dependent matrix multiplication, well matched to parallel accelerators.

Attention weights are learned routing coefficients, not necessarily faithful explanations. Dense attention does not inherently enforce syntax, causality, or factual grounding. Nevertheless, the combination of self-attention, residual paths, normalization, positionwise nonlinear computation, and positional information becomes the substrate for later pretrained language models.

5.5 Synthesis: From Compression to Addressable Context

The four papers progressively relax representational bottlenecks. Sequence-to- sequence learning compresses the source into one state; additive attention makes all encoder states addressable; BPE makes the vocabulary compositionally open; and the Transformer makes addressable interaction the primary computation throughout the model. The common theme is shortening the path between related symbols while retaining a differentiable likelihood objective.

6 The First Pre-Trained Transformers

6.1 Improving Language Understanding by Generative Pre-Training (Link)

Two-stage transfer learning

Radford and coauthors propose a common Transformer that is first trained on a large unlabeled corpus and then adapted to labeled tasks. For tokens $u_1, \ldots, u_T$ and context window k, generative pretraining maximizes

$$L_1(\theta) = \sum_t \log p_\theta(u_t \mid u_{t-k}, \ldots, u_{t-1}).$$

The model is a decoder-style Transformer: causal masking ensures that the representation at position t depends only on tokens through t. This factorization defines a normalized joint distribution

$$p_\theta(u_{1:T}) = \prod_{t=1}^{T} p_\theta(u_t \mid u_{<t}),$$

so the training signal is dense—nearly every token supplies a target.

For a supervised example (x, y), a task head reads the final Transformer representation h_m and predicts, for example,

$$p(y \mid x) = \text{softmax}(W_y h_m).$$

All pretrained parameters are fine-tuned, not frozen. The paper optionally retains language modeling as an auxiliary objective:

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda L_1(\mathcal{C}),$$

where L_2 is supervised log-likelihood. This can stabilize transfer by discouraging movement that destroys the pretrained language model.

Task-aware input transformations

Instead of building a new architecture for every benchmark, the method converts structured inputs into token sequences using delimiter tokens:

- classification uses one document sequence;
- textual entailment concatenates premise and hypothesis;
- similarity processes the two possible orders and combines their representations; and
- multiple-choice tasks concatenate context, question, and each candidate answer, scoring each resulting sequence.

These transformations make a single pretrained computation graph usable across heterogeneous tasks while adding only a small output head. They are early examples of expressing a task through text-like structure rather than designing an entirely task-specific network.

Statistical interpretation

Pretraining provides a data-dependent initialization θ_0. Fine-tuning then seeks a nearby parameter vector that explains a smaller labeled sample. A local quadratic approximation motivates the regularized view

$$\widehat{\theta} \approx \arg\min_\theta \widehat{R}_{\text{sup}}(\theta) + \frac{1}{2}(\theta - \theta_0)^\top \Lambda (\theta - \theta_0),$$

where Λ represents directions strongly constrained by pretraining. The actual algorithm does not explicitly use this penalty, but initialization, small learning rates, and limited fine-tuning implicitly favor parts of the pretrained basin.

The model establishes state-of-the-art results on most of the twelve evaluated tasks and exhibits useful zero-shot linguistic behavior before fine-tuning. Its causal representation is naturally suited to generation, but a token cannot use right context within the same sequence. BERT's objective is designed to remove precisely that restriction for representation learning.

6.2 BERT: Pre-Training for Language Understanding (Link)

Bidirectional Transformer encoding

Devlin and coauthors use an encoder Transformer in which every position attends to both left and right context. Inputs combine WordPiece token, segment, and position embeddings. A special `[CLS]` token supplies a sequence-level representation, while `[SEP]` marks boundaries between segments.

BERT is pretrained with two objectives. The masked-language-model (MLM) loss is

$$L_{\mathrm{MLM}}(\theta) = -\mathbb{E}_{x,M} \sum_{i \in M} \log p_\theta(x_i \mid x_{\setminus M}),$$

where M is a randomly selected set containing 15% of token positions. Among selected positions, 80% are replaced by `[MASK]`, 10% by a random token, and 10% remain unchanged. This mixture gives a supervised signal at selected positions while reducing the discrepancy caused by a symbol that never appears during fine-tuning.

The next-sentence-prediction (NSP) task receives segments (A, B) and predicts whether B actually followed A in the corpus or was sampled randomly:

$$L_{\mathrm{pre}} = L_{\mathrm{MLM}} + L_{\mathrm{NSP}}.$$

The paper's ablations credit NSP on some pairwise tasks, although later models often remove it or replace it with different sentence-order objectives.

MLM is not an autoregressive joint likelihood

An autoregressive model chooses an ordering and defines one joint distribution by the chain rule. BERT predicts selected variables from corrupted versions of the rest. Its loss resembles a stochastic pseudo-likelihood,

$$\sum_i \log p_\theta(x_i \mid x_{-i}),$$

but simultaneous masking means the conditioning context may omit several tokens. The family of learned conditionals need not correspond to a unique, globally consistent normalized joint distribution. BERT is therefore naturally an encoder and conditional representation model, not a left-to-right generator.

Only selected positions directly contribute output loss on each presentation, making MLM less sample-efficient per sequence than causal language modeling. Its compensation is information access: every hidden layer can integrate both directions, which is valuable for token classification and extractive question answering.

Fine-tuning interface and scale

The same pretrained parameters initialize each downstream model. Classification adds a head over `[CLS]`; token labeling adds positionwise heads; and extractive question answering predicts start and end positions. $\text{BERT}_{\text{BASE}}$ uses 12 layers, width 768, 12 attention heads, and about 110 million parameters; $\text{BERT}_{\text{LARGE}}$ uses 24 layers, width 1024, 16 heads, and about 340 million. Pretraining uses BooksCorpus and English Wikipedia.

The paper shows that increasing model size improves downstream accuracy even when labeled task data are modest, and achieves leading results across GLUE, SQuAD, and SWAG. Its central experimental result is architectural economy: one pretrained encoder can be adapted with minimal task-specific machinery.

Limitations include expensive pretraining, quadratic attention, mismatch from mask corruption, a fixed WordPiece vocabulary, and sensitivity of small-data fine-tuning. Bidirectional context also allows correlations unavailable in a causal deployment setting, so performance must be interpreted relative to the information permitted at inference.

6.3 Language Models Are Unsupervised Multitask Learners (Link)

Language modeling as implicit multitask learning

Radford and coauthors argue that web text contains natural demonstrations of many tasks: questions followed by answers, passages followed by summaries, and sentences paired with translations. If task, input, and output can all be expressed as text, a general conditional task model

$$p(\text{output} \mid \text{input}, \text{task})$$

can be embedded inside ordinary sequence likelihood. A sufficiently capable language model may infer the latent task from textual context without explicit task labels or gradient updates.

GPT-2 therefore keeps the causal objective

$$L(\theta) = -\sum_{t} \log p_\theta(x_t \mid x_{<t})$$

and evaluates downstream tasks in a zero-shot setting using natural-language formats and task-specific scoring rules, but no parameter updates. Here "unsupervised" refers to the absence of manually constructed task labels in training; the next-token targets are, of course, observed supervision supplied by the corpus itself. *Self-supervised* is the clearer modern term.

Data, tokenization, and architecture

The authors construct WebText from outbound links on Reddit posts receiving at least three karma, remove Wikipedia documents to reduce overlap with common benchmarks, and obtain roughly eight million documents and 40 GB of text. This heuristic favors pages that humans found interesting, but imports the selection biases, duplication, errors, and social biases of web data.

GPT-2 uses byte-level BPE. Beginning from bytes ensures that any Unicode string can be encoded without an unknown token, while merges compress frequent byte sequences. The paper prevents merges across broad character categories to avoid pathological tokens. Byte-level coverage solves the engineering problem of an open vocabulary; it does not ensure linguistically meaningful segmentation.

The Transformer is decoder-only and includes modifications such as moving layer normalization to the input of each subblock and adding a final layer norm. Four sizes are trained, culminating in a 1.5-billion-parameter model. Increasing model size generally improves both language-model likelihood and zero-shot task performance.

Zero-shot task scoring

Different benchmarks are recast as likelihood comparisons. For a multiple- choice answer a and context c, a basic score is

$$s(a; c) = \sum_{t=1}^{|a|} \log p_\theta(a_t \mid c, a_{<t}).$$

Because raw log-likelihood accumulates with length and frequent strings have large unconditional probability, evaluations may require length normalization or calibration against an unconditional score. Such choices are part of the zero-shot method and can materially affect results.

For reading comprehension, summarization, translation, question answering, and other tasks, the paper supplies prompts or formatting cues and measures the continuation. Results range from weak to competitive or state of the art, depending on the task. The important pattern is consistent improvement with scale, not uniform mastery.

Generalization, contamination, and interpretation

Zero-shot success can arise through several mechanisms: learning a reusable algorithm, recognizing a task format, retrieving memorized content, or interpolating among similar demonstrations in pretraining. These mechanisms cannot be separated by benchmark scores alone. The paper investigates overlap between WebText and evaluation sets, anticipating the contamination problem that becomes more severe as corpora scale.

The report does not establish that next-token prediction inevitably yields a universal learner. Its largest model still underfits WebText and fails on many tasks. It provides early evidence for an empirical scaling hypothesis: broader data and larger causal Transformers produce emergent-looking transfer without task-specific training, motivating later work on in-context learning and scaling laws.

6.4 Synthesis: Three Transfer Regimes

The papers define three distinct uses of pretraining:

1. GPT learns a causal model, then updates all parameters on labeled data.

2. BERT learns bidirectional conditional representations, then fine-tunes a small task interface together with the encoder.
3. GPT-2 attempts to specify the task in text and performs it with no parameter updates.

Fine-tuning adapts strongly but requires task data and risks forgetting; bidirectional masking supplies rich encodings but not a direct generative joint; zero-shot prompting maximizes reuse but makes performance depend on how the task is verbalized. These are complementary choices about objective, information flow, and where adaptation occurs.

7 Scaling Up

7.1 The Bitter Lesson (Link)

Historical claim

Sutton's essay argues that the most durable advances in artificial intelligence have come from general methods that exploit increasing computation, especially search and learning. Researchers repeatedly encode human domain knowledge to obtain an immediate advantage; as compute grows, systems built around scalable computation eventually overtake them. The lesson is "bitter" because expert knowledge is intellectually attractive and can improve current systems, yet may constrain the methods that dominate in the long run.

The essay identifies recurring examples:

- chess shifted from attempts to encode human chess knowledge toward massive search;
- Go resisted brute-force methods longer, but search combined with learning and self-play ultimately surpassed handcrafted approaches;
- speech recognition moved from linguistically structured pipelines toward statistical learning from data; and
- computer vision moved from handcrafted features and explicit geometric structures toward learned representations.

The positive prescription is not "computation alone." It is to design methods whose performance can continue improving as more computation and experience become available. Search supplies scalable deliberation; learning supplies scalable acquisition of regularities.

A formal lens on scalability

Let method A have performance $P_A(C)$ at compute budget C. A handcrafted method may start ahead but saturate,

$$\lim_{C\to\infty} P_A(C) = P_A^*,$$

whereas a general method B may initially be worse yet satisfy $P_B(C) > P_A(C)$ beyond some crossover budget. Choosing only by performance at today's C can then select the wrong long-run research direction.

This is a useful abstraction, not a theorem asserted by the essay. Performance also depends on data, algorithms, hardware efficiency, and the task distribution. No finite historical sample proves that every scalable method must eventually win.

Nuanced reading

The lesson opposes building in the *contents* of human solutions when that prevents scale; it does not eliminate inductive bias. Convolution, attention, tokenization, objectives, augmentation, and search spaces all encode researcher judgment. The sharper question is whether an assumption helps the system learn more effectively and remains useful as resources grow, or hard-codes a brittle ceiling.

Scaling can also amplify undesirable data regularities, cost, and concentration of access. A method's ability to consume compute is not evidence of safety, causal understanding, or social value. Sutton's essay is a methodological forecast about capability research, not a complete criterion for choosing AI systems.

7.2 Language Models Are Few-Shot Learners (Link)

In-context learning

Brown and coauthors train GPT-3, a 175-billion-parameter causal Transformer, and evaluate tasks without gradient updates. A prompt contains an optional task description and demonstrations

$$(x_1, y_1), \ldots, (x_k, y_k), x_*;$$

the model predicts y_* as a continuation. The paper distinguishes:

zero-shot a task instruction and no demonstrations;

one-shot one demonstration; and

few-shot several demonstrations fitting within the context window.

The model parameters remain fixed. "Learning" here means that the conditional computation changes with the prompt, not that an optimizer updates θ:

$$p_\theta(y_* \mid x_*, (x_1, y_1), \ldots, (x_k, y_k)).$$

This distinction separates in-context learning from ordinary few-shot fine-tuning.

Likelihood-based decision rules

For candidate answer a, a common score is

$$s(a) = \sum_{t=1}^{|a|} \log p_\theta(a_t \mid P, a_{<t}),$$

where P is the prompt. Raw likelihood confounds task compatibility with answer frequency and length. The paper therefore uses task-dependent normalization in some evaluations, including comparison with unconditional likelihood. A generic calibrated score is

$$s_{\text{cal}}(a) = \log p_\theta(a \mid P) - \lambda \log p_\theta(a \mid P_0),$$

where P_0 removes task-specific evidence. Prompt format, example order, and scoring are part of the estimator, even though no weights are trained.

Scale and empirical behavior

The study trains a family from 125 million to 175 billion parameters on a mixture of filtered Common Crawl, WebText-like data, books, and Wikipedia. Across many tasks, zero-, one-, and few-shot performance improve with model size; the advantage from demonstrations often grows as models become larger. GPT-3 is strong on several question-answering, translation, cloze, and synthetic reasoning tasks, but remains weak on others, including some natural-language inference and reading-comprehension benchmarks.

The results support a meta-learning interpretation: next-token training across diverse text may internalize a procedure that infers tasks from demonstrations. They do not identify the procedure or show that every improvement is a new algorithm. Pattern matching, retrieval, format recognition, and genuine rule induction can yield the same aggregate score.

Contamination, bias, and scope

At web scale, test examples or close paraphrases may occur in training. The authors perform overlap analyses and flag affected results, but exact auditing is difficult. Memorization is particularly concerning for small, publicly available benchmarks. Reliable claims require deduplication, temporally new tests, and evaluations designed after training.

The paper also documents biased associations, uneven performance across groups, energy use, misuse potential, and unreliable generation. Scaling likelihood training improves broad task performance but does not align the model with human intent or make its confidence calibrated. These limitations motivate the later shift from pretrained model to aligned assistant.

7.3 Scaling Laws for Neural Language Models (Link)

Empirical power laws

Kaplan and coauthors measure cross-entropy loss as model parameters N, data tokens D, and training compute C vary. Over the studied regimes, held-out loss is well described by power laws approaching an irreducible floor:

$$L(N) \approx L_\infty + A_N N^{-\alpha_N}, \qquad L(D) \approx L_\infty + A_D D^{-\alpha_D},$$

and along efficient compute frontiers,

$$L(C) \approx L_\infty + A_C C^{-\alpha_C}.$$

On log–log axes, the reducible loss is approximately linear:

$$\log(L - L_\infty) = \log A - \alpha \log X.$$

The exponent is an elasticity: a multiplicative increase $X \mapsto rX$ reduces the fitted excess loss by $r^{-\alpha}$.

These are empirical regularities, not asymptotic theorems. Exponents depend on the data distribution, tokenizer, architecture family, optimizer, and range of scale used in the fit. Extrapolating several orders of magnitude assumes the same regime persists.

Compute, data, and parameter bottlenecks

For dense Transformers, a useful training approximation is

$$C \approx \kappa N D,$$

with κ accounting for forward and backward operations per parameter and token. A model can be parameter-limited, data-limited, or compute-limited. Increasing N with too little data leaves parameters undertrained; repeatedly cycling a small data set eventually yields diminishing returns and overfitting.

The paper finds that performance depends weakly on many architectural details once total non-embedding parameter count is controlled, within the investigated Transformer family. Larger models are more sample-efficient: they reach a given loss after fewer optimization steps and fewer tokens, though each step costs more.

Kaplan compute-optimal prescription

Under a fixed compute budget, training a model all the way to convergence is not generally optimal; one should use a larger model for fewer steps. The paper's fitted frontier allocates additional compute predominantly to parameters, with approximate scalings often summarized as

$$N_{\text{opt}} \propto C^{0.73}, \qquad D_{\text{opt}} \propto C^{0.27}.$$

These exponents are estimates tied to the experimental protocol, particularly the treatment of learning-rate schedules and partially trained runs. They are not universal constants. Chinchilla revisits the allocation with a different experimental design and obtains a substantially more data-intensive optimum.

Implications and limits

Smooth loss scaling makes training outcomes forecastable and supports rational allocation of large compute budgets. Yet average token loss is not a complete measure of capability. Downstream accuracy may have thresholds, nonmonotonicity, or metric saturation; rare capabilities and safety failures can be invisible in mean cross-entropy. Scaling laws predict performance within a modeled regime, not which social or cognitive properties must emerge.

7.4 Training Compute-Optimal Large Language Models (Link)

Re-estimating the compute frontier

Hoffmann and coauthors argue that prominent large language models were undertrained: their parameter counts grew much faster than the number of training tokens. Using more than 400 training runs, they estimate the allocation of a fixed compute budget between parameters N and tokens D through three complementary approaches: envelope fitting, iso-compute curves, and a parametric loss model.

The parametric form is

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}},$$

where E is irreducible data entropy, the second term captures finite-model error, and the third captures finite-data error. Under $C = \kappa ND$, optimization makes the two reducible terms balance at a compute-dependent ratio.

Proposition 7.1 (Optimal allocation in the separable power-law model)**.** *Under the loss and compute models above,*

$$N_{\text{opt}} \propto C^{\beta/(\alpha+\beta)}, \qquad D_{\text{opt}} \propto C^{\alpha/(\alpha+\beta)}.$$

Proof. Substitute $D = C/(\kappa N)$ into the reducible loss:

$$R(N) = AN^{-\alpha} + B(\kappa N/C)^{\beta}.$$

Setting $R'(N) = 0$ gives $N^{\alpha+\beta} \propto C^{\beta}$. Substitution into the compute constraint gives the expression for D. □

The fitted exponents are close enough that both parameters and tokens scale approximately as $C^{1/2}$: when model size doubles, the training-token count should also roughly double. Since $C \propto ND$, doubling both requires about four times the compute.

The Chinchilla experiment

For approximately the compute used to train the 280-billion-parameter Gopher model, the fitted frontier predicts a much smaller model trained on substantially more data. The authors train Chinchilla with 70 billion parameters on 1.4 trillion tokens. It outperforms larger models on average across a wide range of language benchmarks while being cheaper at inference.

This is an unusually strong test of a scaling prescription: the paper does not only fit small experiments but trains a frontier-scale model predicted by the fit. The result shifts practice from maximizing parameter count toward balancing model capacity with data exposure.

Reconciling Kaplan and Chinchilla

Both papers agree that fixed compute should not be spent training a small model to complete convergence. They disagree quantitatively about how rapidly N and D should grow. Chinchilla attributes much of the difference to fitting runs whose learning-rate schedules were designed for a longer token horizon; an intermediate checkpoint from such a run overstates the loss attainable by a schedule optimized to end at that checkpoint. Chinchilla also studies more large-model runs and fits joint dependence on N and D.

The "about 20 tokens per parameter" summary is a rule of thumb for the paper's data, architecture, and training regime, not a physical constant. Data quality, repetition, optimizer efficiency, sparsity, inference demand, and downstream adaptation all alter the economically optimal choice. When a model will serve many queries, a smaller, more thoroughly trained model may be preferable even beyond the training-compute optimum because inference cost scales with N.

7.5 Synthesis: Scaling as an Experimental Science

Sutton supplies the historical prior that general learning and search improve with computation. GPT-3 shows that scale can turn a single likelihood-trained model into an in-context task learner. Kaplan makes loss predictable through empirical power laws, and Chinchilla shows that the allocation of compute is as important as its amount. Together they replace "make the model larger" with a more precise program: measure scaling surfaces, balance parameters and data, validate extrapolations at scale, and distinguish lower average loss from the particular capabilities or risks ultimately of interest.

8 Deep Reinforcement Learning

8.1 Human-Level Control through Deep Reinforcement Learning (Link)

From pixels to action values

Mnih and coauthors combine Q-learning with a convolutional network to learn Atari games directly from image pixels. In a discounted Markov decision process with transition kernel P, reward r, and discount $0 \le \gamma < 1$, the optimal action-value function satisfies the Bellman equation

$$Q^*(s,a) = \mathbb{E}\Big[r + \gamma \max_{a'} Q^*(s',a') \mid s,a\Big].$$

The greedy policy $\pi^*(s) \in \arg\max_a Q^*(s,a)$ is optimal when Q^* is known. DQN represents all action values for a state with one network $Q(s,a;\theta)$ whose convolutional features are learned from reward.

Proposition 8.1 (Bellman optimality contraction)**.** *For bounded action-value functions and the sup norm, the Bellman optimality operator*

$$(TQ)(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q(s',a') \mid s,a]$$

satisfies $\|TQ - TQ'\|_\infty \le \gamma \|Q - Q'\|_\infty$*. Consequently it has a unique fixed point* Q^**.*

Proof. Expectation is nonexpansive and $|\max_a q_a - \max_a q'_a| \le \max_a |q_a - q'_a|$. Multiplication by γ gives the inequality; the fixed-point conclusion follows from the Banach contraction theorem. □

This tabular convergence intuition does not transfer automatically to nonlinear function approximation with off-policy data and bootstrapped targets—the combination sometimes called the deadly triad.

The DQN loss and stabilizers

DQN stores transitions $e_t = (s_t, a_t, r_t, s_{t+1})$ in replay memory and samples mini-batches approximately uniformly. At iteration i it minimizes

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(y_i - Q(s,a;\theta_i)\right)^2\right],$$

$$y_i = r + \gamma \max_{a'} Q(s',a';\theta_i^-),$$

where target parameters θ_i^- are copied from the online parameters only periodically. The update uses a semi-gradient: y_i is treated as constant while differentiating with respect to θ_i.

Replay breaks short-range temporal correlations, reuses expensive experience, and mixes data from several recent policies. The target network slows movement of the regression target. Neither device restores independent sampling or proves convergence, but together they make deep Q-learning empirically stable. An ϵ-greedy behavior policy supplies exploration.

The input stacks four preprocessed frames so velocity and direction can be inferred; rewards are clipped to control scale. One architecture and hyperparameter set is used across 49 games, though a separate network is trained for each. DQN exceeds prior agents on most games and reaches or surpasses a professional tester's normalized score on many, but struggles on games needing long exploration, precise planning, or memory. Score normalization and restricted evaluation time also make "human-level" an aggregate benchmark claim, not human-like general intelligence.

8.2 Mastering the Game of Go with Deep Neural Networks and Tree Search (Link)

Policy, value, and search

Silver and coauthors build AlphaGo from three complementary approximations:

- a supervised policy $p_\sigma(a \mid s)$ trained to predict expert moves;
- a reinforcement-learned policy $p_\rho(a \mid s)$ improved through self-play to maximize winning probability;
- a value network $v_\theta(s)$ trained to predict the winner from a board position.

A fast, lower-capacity rollout policy supplies inexpensive simulations. Monte Carlo tree search (MCTS) combines policy priors, stored action values, visit counts, neural value estimates, and rollout outcomes.

The supervised policy maximizes

$$\sum_{(s,a)\in\mathcal{D}_{\text{expert}}} \log p_\sigma(a \mid s).$$

High move-prediction accuracy is useful but not identical to winning: several moves may be strong, and a common expert move need not maximize match outcome. The RL policy therefore optimizes terminal reward $z \in \{-1, +1\}$ with a REINFORCE-style gradient

$$\nabla_\rho J(\rho) = \mathbb{E}_{\tau\sim p_\rho}\left[z \sum_t \nabla_\rho \log p_\rho(a_t \mid s_t)\right].$$

It is initialized from the supervised policy and plays against randomly sampled earlier versions, reducing cycling against only the current opponent.

Learning a value from self-play

The value network minimizes

$$\mathbb{E}_{(s,z)}(z - v_\theta(s))^2$$

on positions generated by self-play with the improved policy. Sampling many adjacent positions from one game would create highly correlated examples with the same outcome; the training construction samples positions across games to reduce this redundancy. The value network amortizes expensive simulation by learning the expected result of future play in one forward pass.

Policy evaluation is distribution dependent:

$$v^\pi(s) = \mathbb{E}_\pi[z \mid s].$$

Changing the self-play policy changes the state distribution and the value being estimated. The network is not an oracle for arbitrary play outside its training distribution.

Neural-guided MCTS

Each tree edge stores a prior $P(s,a)$, visit count $N(s,a)$, and mean action value $Q(s,a)$. A simulation selects an action maximizing an exploitation term plus a prior-weighted exploration bonus,

$$a_t = \arg\max_a \{Q(s_t,a) + u(s_t,a)\},$$

where u decreases as an edge is visited and increases with its policy prior. At a leaf, evaluation mixes the value-network prediction with the result of a fast rollout. That estimate is backed up along the path, updating counts and means. Search returns a stronger policy than either network alone because it allocates computation adaptively to the current position.

AlphaGo defeats the European champion Fan Hui five games to zero and achieves dominant results against prior Go programs. The scientific lesson is hybrid: learning supplies priors and approximate evaluation, while search performs test-time policy improvement. The system is specialized to Go, uses known rules and extensive self-play, and should not be described as learning solely from raw experience or as a domain-general reasoner.

8.3 Deep Reinforcement Learning from Human Preferences (Link)

Replacing a programmed reward

Christiano and coauthors address tasks for which the desired behavior is easier to recognize than to encode as a reward. The agent alternates among three processes:

1. a policy generates trajectories under the current learned reward;
2. a human compares short pairs of trajectory segments; and
3. a reward predictor is trained from those comparisons and returned to the policy optimizer.

Atari policies use advantage actor–critic, while simulated-control policies use trust-region policy optimization. Feedback and policy learning remain interleaved because the most informative comparisons change as behavior improves.

Preference likelihood

For a segment $\sigma = ((o_1,a_1),\dots,(o_k,a_k))$, define predicted utility

$$R_\phi(\sigma) = \sum_{t=1}^{k} \widehat{r}_\phi(o_t,a_t).$$

The Bradley–Terry/Luce choice model assigns

$$P_\phi(\sigma^1 \succ \sigma^2) = \frac{\exp R_\phi(\sigma^1)}{\exp R_\phi(\sigma^1) + \exp R_\phi(\sigma^2)} = \sigma\Big(R_\phi(\sigma^1) - R_\phi(\sigma^2)\Big).$$

If a label distribution $\mu \in \Delta^2$ records preference, indifference, or noisy judgment, the reward model minimizes cross-entropy

$$-\mu_1 \log P_\phi(\sigma^1 \succ \sigma^2) - \mu_2 \log P_\phi(\sigma^2 \succ \sigma^1).$$

Equal preference uses $\mu = (1/2, 1/2)$; incomparable clips are discarded.

Only utility differences are identified. Adding a constant c to every per-step reward adds the same kc to equal-length segments and leaves all preference probabilities unchanged. Multiplying rewards changes the effective choice temperature, so scale is entangled with the assumed noise model. The learned reward is a latent explanatory score, not a directly observed human utility.

Active queries and distribution shift

An ensemble of reward predictors is trained on bootstrapped preference data. Candidate segment pairs are ranked by disagreement among ensemble predictions, as a rough uncertainty-sampling rule. This can reduce labeling cost, although ensemble variance is not a calibrated posterior and the paper finds that active selection does not help uniformly.

The reward model is trained on trajectories produced by evolving policies. Optimizing it pushes the policy toward states where the predictor has little data, creating a feedback loop:

$$\text{reward error} \to \text{policy exploitation} \to \text{new state distribution}.$$

Continued human queries can correct some errors. Training the reward model once offline performs poorly because it cannot anticipate all ways the policy will exploit its extrapolations.

Results and alignment lesson

With minutes to hours of nonexpert comparison time—far less than the number of environment interactions—agents learn locomotion and Atari behaviors that are often competitive with direct reward training. Human feedback sometimes prefers qualitatively desirable behavior omitted from the programmed reward, but short clips can hide delayed consequences and make some tasks difficult.

The approach reduces reward engineering; it does not eliminate specification. Outcomes depend on instructions to raters, which clips are shown, aggregation of disagreement, reward-model capacity, and the policy optimizer. A powerful agent can exploit systematic mistakes in $\widehat{r}_\phi$. The paper establishes the core RLHF decomposition—learn a reward from comparisons, then optimize a policy against it—while also exposing reward-model overoptimization as a central safety problem.

8.4 Synthesis: Learning Values, Planning, and Learning the Objective

DQN learns an action-value function from environmental reward; AlphaGo combines learned policies and values with search; preference-based RL learns the reward signal itself from

human judgments. Moving the learned boundary upward increases flexibility but also changes the dominant error: value approximation and bootstrapping in DQN, search-distribution mismatch in AlphaGo, and objective misspecification in preference learning. In all three systems, replay, self-play, search, or active feedback controls the distribution of data on which the next model is trained.

9 From Model to Assistant

9.1 Training Language Models to Follow Instructions with Human Feedback (Link)

Alignment pipeline

Ouyang et al. turn a pretrained language model into an assistant in three stages. First, supervised fine-tuning (SFT) maximizes the likelihood of contractor-written demonstrations. Second, contractors rank several answers to the same prompt and a scalar reward model (RM) is fit to those comparisons. Finally, proximal policy optimization (PPO) optimizes the language model against the learned reward while constraining it to remain near the SFT model. Thus the data specify desired behavior indirectly: demonstrations identify a useful region of policy space, while comparisons resolve finer preferences.

For a prompt x, preferred answer y_w, and rejected answer y_l, the RM is fit with the Bradley–Terry likelihood

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma(r_\theta(x, y_w) - r_\theta(x, y_l)) \,,$$

and hence minimizes

$$\mathcal{L}_{\mathrm{RM}}(\theta) = -\mathbb{E} \log \sigma(r_\theta(x, y_w) - r_\theta(x, y_l)) \,.$$

Only reward differences are identifiable. Adding any prompt-dependent constant $c(x)$ to every reward leaves the comparison probabilities unchanged.

The policy objective is approximately

$$\max_{\phi} \; \mathbb{E}_{x,, y \sim \pi_\phi(\cdot|x)} \left[r_\theta(x, y) - \beta \log \frac{\pi_\phi(y \mid x)}{\pi_{\mathrm{SFT}}(y \mid x)} \right] .$$

The second term is a sampled KL penalty: it discourages reward hacking and catastrophic drift into text on which the RM is unreliable. The paper's PPO-ptx variant also mixes in the original next-token objective, preserving general capabilities that instruction tuning might otherwise degrade.

What the experiments establish

Human evaluators prefer the 1.3B-parameter InstructGPT model to the 175B-parameter base GPT-3 model on the prompt distribution used in the study. The result separates scale from intent-following: next-token pretraining alone does not determine whether a model is

helpful in interaction. The aligned models also improve truthfulness and reduce toxic output on several tests, though no stage guarantees honesty or harmlessness.

The target is not "human values" in the abstract. It is the preferences of a particular contractor pool, elicited by particular instructions and evaluated on a particular prompt distribution. Reward-model error, annotator disagreement, distribution shift, and optimization against an imperfect proxy remain distinct failure modes. In particular, maximizing an RM beyond its training support can increase its score while reducing actual answer quality.

9.2 Direct Preference Optimization: Your Language Model Is a Reward Model (Link)

Eliminating the explicit reward model

Rafailov et al. begin with the KL-regularized control problem

$$\max_{\pi} \ \mathbb{E}_{x\sim\mathcal{D},,y\sim\pi(\cdot|x)}[r(x,y)] - \beta\mathbb{E}_{x\sim\mathcal{D}}\,\mathrm{KL}(\pi(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x))\,.$$

For each fixed x, a Lagrange-multiplier calculation gives

$$\pi_r(y \mid x) = \frac{\pi_{\mathrm{ref}}(y \mid x)e^{r(x,y)/\beta}}{Z(x)}, \qquad Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x)e^{r(x,y)/\beta}.$$

Equivalently,

$$r(x,y) = \beta \log \frac{\pi_r(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x).$$

The last term depends only on the prompt, so it cancels from every Bradley–Terry reward difference. Substitution therefore converts preference learning directly into a policy loss:

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right).$$

Thus the log policy ratio acts as an implicit reward. DPO requires neither a separately deployed RM nor on-policy RL rollouts; it is an ordinary classification-style optimization over a fixed preference dataset.

Gradient interpretation and qualifications

Writing the bracketed logit as s_θ, differentiation gives a weight $\sigma(-s_\theta)$ multiplying a gradient that raises the preferred response's log likelihood relative to the rejected response. Already well-ordered pairs receive little weight; violated or uncertain pairs receive more. The reference model is essential: it determines the baseline against which these relative changes are measured, while β controls the strength of departure.

DPO is an exact reparameterization only under the assumed KL-regularized objective and preference model. In practice it is offline, inherits label noise and coverage gaps, cannot explore to obtain informative new comparisons, and can overfit preference shortcuts. Simplicity removes PPO's instability; it does not remove objective misspecification or distribution shift.

9.3 Chain-of-Thought Prompting Elicits Reasoning in Large Language Models (Link)

Prompting computation into the context

Wei et al. augment each few-shot exemplar with a natural-language sequence of intermediate steps. If $z = (z_1, \ldots, z_m)$ is a reasoning trace and y the answer, autoregressive generation realizes

$$p_\theta(z, y \mid x) = \prod_{t=1}^{m} p_\theta(z_t \mid x, z_{<t})\, p_\theta(y \mid x, z).$$

The trace provides additional serial computation and exposes intermediate variables in the token context. No parameter update occurs: the behavior is elicited by the demonstration format at inference time.

The paper reports strong gains on arithmetic, commonsense, and symbolic tasks, with the most striking results at large model scale. Small models often produce incoherent traces or even lose accuracy, suggesting an emergent capability rather than a universally useful prompt template. On GSM8K, a sufficiently large prompted model closes much of the gap to task-specific finetuning while retaining a single general interface.

What a chain of thought does not prove

An emitted rationale is an observable computational artifact, not necessarily a faithful causal account of the model's internal computation. A model may reach a correct answer through an invalid step, rationalize a guess, or change its answer under superficial prompt changes. Greedy chain-of-thought prompting also differs from later *self-consistency*, which samples multiple traces and aggregates their answers. The paper establishes that intermediate text can improve task performance; it does not establish formal correctness, faithfulness, or robust planning.

9.4 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Link)

Parametric and nonparametric memory

Lewis et al. combine a pretrained sequence-to-sequence generator with a dense retriever over a Wikipedia index. A query encoder $q_\eta(x)$ and document embeddings $d(z)$ define, over an approximate top-K set,

$$p_\eta(z \mid x) = \frac{\exp(d(z)^\top q_\eta(x))}{\sum_{z' \in \operatorname{top} K(x)} \exp(d(z')^\top q_\eta(x))}.$$

The generator's weights are parametric memory; the retrievable corpus is nonparametric memory that can be inspected or replaced without retraining the entire generator.

RAG-Sequence chooses one latent document for the whole output and marginalizes it:

$$p(y \mid x) = \sum_{z \in \text{top } K(x)} p_\eta(z \mid x) \prod_{i=1}^{|y|} p_\theta(y_i \mid x, z, y_{<i}).$$

RAG-Token permits the latent document to change at every output position:

$$p(y \mid x) = \prod_{i=1}^{|y|} \sum_{z \in \text{top } K(x)} p_\eta(z \mid x) p_\theta(y_i \mid x, z, y_{<i}).$$

The first encourages document-level consistency; the second is more flexible when different portions of an answer require different evidence. Because the document is latent, end-to-end likelihood training can improve retrieval even without document supervision.

Evidence, approximation, and failure

Retrieval improves open-domain QA and produces more specific, diverse text on knowledge-intensive generation tasks. It also creates a potential provenance path from a claim to a source. But top-K search is a discrete approximation: documents excluded by retrieval receive no generator gradient. An answer can fail because the corpus lacks the fact, the index or query encoder misses it, the passage is stale, or the generator ignores or misuses good evidence. Retrieval therefore improves access to facts; it is not a factuality theorem.

9.5 ReAct: Synergizing Reasoning and Acting in Language Models (Link)

An interleaved agent trajectory

Yao et al. place natural-language reasoning and environment interaction in one autoregressive trajectory:

$$c_t = (x, \tau_{<t}), \qquad a_t \sim \pi_\theta(\cdot \mid c_t), \qquad o_t \sim P(\cdot \mid a_t, c_t).$$

Here an a_t token block may be a *Thought*, which changes only the model's context, or an *Action*, which invokes an external interface; the environment then appends an *Observation*. A thought can decompose a goal, identify what evidence is missing, and interpret the returned result. An observation can correct an unsupported thought before it propagates.

On HotpotQA and FEVER the actions search and inspect Wikipedia; on ALFWorld and WebShop they manipulate an interactive environment. Few-shot ReAct prompts often outperform action-only trajectories because reasoning makes the policy's state explicit, and outperform chain-of-thought-only prompting because new observations ground the reasoning. The paper also finds complementarity: falling back between ReAct and chain of thought can exceed either alone.

Agentic failure modes

The method introduces errors at every interface. The model may formulate a bad query, select the wrong action, misread an observation, loop, stop early, or invent a rationale inconsistent with its evidence. Tool access changes the information set but does not ensure correct use of information, and visible thoughts remain imperfect explanations. ReAct's central contribution is the closed loop: reason to choose an action, observe its consequence, and revise.

9.6 Why Language Models Hallucinate (Link)

Pretraining as an estimation problem

Kalai et al. argue that hallucination is not only a consequence of noisy data or insufficient alignment. Even with clean data, a learner must estimate which strings are valid from finite observations. They reduce generation to an "is-it-valid" discrimination problem and connect unavoidable errors on rare facts to the classical missing-mass problem: facts seen once—the singletons—signal probability mass on valid facts not seen at all. In the paper's formal setting, this yields lower bounds linking a base model's error to the singleton rate. More parameters cannot recover distinctions that the sample does not statistically identify.

This perspective separates two questions. Pretraining explains why a model assigns probability to plausible false statements; post-training determines whether the assistant states them confidently, qualifies them, or abstains. Retrieval and better data can reduce the unobserved mass but do not generally make it zero.

Why common evaluations reward guessing

The paper's second thesis is incentive-based. Standard benchmarks award a point for an exact correct answer and treat both an incorrect answer and "I don't know" as zero. Whenever a model gives some answer positive probability of being correct, guessing weakly dominates abstention. A simple selective prediction calculation makes the issue explicit. Give reward 1 for a correct answer, penalty $-\lambda$ for a wrong answer, and 0 for abstention. If the model's probability of correctness is q, answering has value

$$q - \lambda(1 - q) = (1 + \lambda)q - \lambda,$$

so it should answer only when

$$q > \frac{\lambda}{1 + \lambda}.$$

Exact accuracy effectively takes $\lambda = 0$, producing a zero confidence threshold. Uncertainty-aware scoring or explicit credit for calibrated abstention changes the optimal behavior.

The practical recommendation is therefore systemic: hallucination-specific tests are insufficient if the primary capability leaderboard still rewards guessing. Evaluations should make uncertainty and refusal part of the score. The analysis does not claim that every false output has one cause, nor that abstention alone solves factuality; it explains why statistical uncertainty arises and why prevailing incentives often conceal rather than express it.

9.7 Synthesis: From a Language Model to an Assistant

These papers modify different parts of the interaction loop. Instruction tuning and RLHF learn behavior from demonstrations and preferences; DPO makes a particular preference objective easier to optimize; chain of thought allocates tokens to intermediate computation; RAG supplies external memory; and ReAct closes the loop between computation and action. None substitutes for the others. A modern assistant may use all five and still hallucinate because its data are incomplete, its reward is misspecified, retrieval fails, reasoning is unfaithful, or an accuracy-only evaluation makes guessing optimal.

10 Reasoning Models

10.1 DeepSeekMath (Link)

Domain pretraining before reinforcement learning

DeepSeekMath separates mathematical capability into three interventions: domain-adaptive pretraining, instruction tuning, and reinforcement learning. DeepSeekMath-Base 7B continues pretraining a code model on 120 billion math-related tokens mined from Common Crawl, mixed with natural-language and code data. The corpus is obtained by iterating between a fastText classifier, web retrieval, and human inspection. Starting from a code model performs better than starting from a general model, consistent with the hypothesis that code pretraining supplies useful representations for formal structure and multi-step execution.

Instruction tuning then mixes several solution modes: chain-of-thought text, program-of-thought solutions, and tool-integrated reasoning. These modes should not be conflated. A textual derivation asks the model itself to execute every step, whereas a program or tool call delegates part of the computation to an external interpreter. The paper's ablations also caution that simply adding nominally mathematical sources is not sufficient: additional arXiv data did not consistently improve the adopted benchmarks.

From PPO to group-relative policy optimization

PPO normally trains a value model V_ψ to construct low-variance advantage estimates. For language models this critic is expensive—it may be comparable in size to the policy—and sequence-level rewards make token-level value estimation awkward. Group-relative policy optimization (GRPO) removes the critic. Given a question q, sample a group

$$o_1, \dots, o_G \overset{\text{iid}}{\sim} \pi_{\theta_{\text{old}}}(\cdot \mid q)$$

and obtain rewards $r_1, \dots, r_G$. For outcome supervision, the normalized group-relative advantage is

$$\widehat{A}_i = \frac{r_i - \bar{r}}{s_r + \delta}, \qquad \bar{r} = \frac{1}{G} \sum_{j=1}^{G} r_j,$$

where s_r is the within-group standard deviation and δ prevents division by zero. All tokens in an output inherit this terminal advantage. The clipped surrogate has the schematic form

$$\mathcal{J}_{\mathrm{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\Big(\rho_{i,t}\widehat{A}_i, \mathrm{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\widehat{A}_i\Big) - \beta D_{\mathrm{KL},i,t}\right\}\right],$$

with

$$\rho_{i,t} = \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\mathrm{old}}}(o_{i,t} \mid q, o_{i,<t})}.$$

The clipping limits a single update's policy change, while the reference-model KL term controls cumulative drift. Group centering is a baseline: conditional on a prompt it compares completions with one another, reducing variance without learning an absolute value function. It also makes the update depend strongly on group diversity. If all sampled completions receive the same reward, their normalized advantages carry essentially no learning signal.

Outcome versus process supervision

Outcome supervision assigns one reward to a completed solution. It is cheap when an answer can be checked automatically, but it cannot identify which step caused success or failure. Process supervision supplies step-level rewards and can define a token's advantage from the rewards of subsequent reasoning steps. The latter gives denser credit assignment but requires a reliable process reward model and much more granular data. DeepSeekMath reports stronger results from process-supervised GRPO in its setting, while the later R1 work emphasizes the scalability of verifiable outcome rewards.

The resulting DeepSeekMath-RL 7B improves from 46.8 to 51.7 percent on MATH under the paper's top-1 textual-reasoning evaluation; self-consistency over 64 samples reaches 60.9 percent. These numbers demonstrate the interaction of specialized data, post-training, and inference-time aggregation rather than a single universally dominant ingredient.

10.2 DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL (Link)

R1-Zero: reasoning induced by verifiable reward

DeepSeek-R1-Zero applies GRPO directly to DeepSeek-V3-Base, without a preceding supervised reasoning stage. For mathematics, code, and logic problems, rewards are principally rule based: correctness is checked against a reference answer or by a compiler/test suite, and formatting can be checked mechanically. This avoids training a neural outcome reward model that might itself be exploited.

For prompt q, the GRPO update estimates whether each sampled answer is better or worse than its peers. Repeated online sampling is important: as the policy changes, it generates new errors and new successful strategies, so the training distribution follows the current policy rather than remaining a fixed offline dataset. Under sufficiently long rollouts, behaviors such

as backtracking, verification, and strategy switching appear even though no human-written reasoning trace specifies them. The celebrated "aha moment" is evidence of an optimization trajectory discovering a useful behavioral pattern, not evidence that the model has acquired human-like introspection.

Final-answer reward creates an identifiability problem. If z is a reasoning trace and y the final answer, training observes a reward such as

$$R(q, z, y) = \mathbf{1}\{y = y^*(q)\},$$

not whether every statement in z is valid. Many traces—including spurious ones—can lead to the same rewarded answer. Outcome RL therefore optimizes successful trajectories in aggregate; it does not certify the faithfulness or logical validity of the displayed chain of thought.

Why pure RL is not the final assistant recipe

R1-Zero develops strong reasoning but also produces poor readability, language mixing, and weak behavior on general assistant tasks. DeepSeek-R1 uses a multi-stage pipeline:

1. a small cold-start SFT set establishes a readable reasoning format;
2. reasoning-oriented GRPO improves verifiable tasks;
3. rejection sampling from the resulting checkpoint creates new reasoning data, which is mixed with general instruction data for SFT;
4. a final RL stage combines reasoning rewards with helpfulness, harmlessness, and language-consistency signals.

This pipeline exposes a real tradeoff. Unconstrained exploration can discover effective computations, whereas supervised and preference data make those computations usable in a general assistant. Cold-start supervision is therefore an interface and distribution-shaping device, not the claimed source of all reasoning capability.

Distillation and empirical interpretation

Long reasoning traces generated by R1 are used to fine-tune smaller dense models. Their improvements show that part of the behavior found through large-scale RL can be transferred by ordinary likelihood training:

$$\mathcal{L}_{\text{distill}}(\phi) = -\mathbb{E}_{(q,z,y)\sim\mathcal{D}_{R1}} \log p_\phi(z, y \mid q).$$

This does not imply that SFT and RL are equivalent. The teacher's dataset was created by online search under reward, while the student only imitates the selected trajectories and cannot discover beyond their support.

Reported gains are largest on tasks with checkable answers, including AIME, MATH, coding, and scientific questions. Pass@k, majority voting, and top-1 accuracy measure different systems: increasing k spends more inference compute and can conceal a weak single-sample

policy. Further limitations include reward hacking, benchmark contamination, high rollout cost, verbosity, and the possibility that longer reasoning merely increases the opportunity to recover from an initially poor path rather than reflecting a monotone increase in reasoning quality.

10.3 s1: Simple Test-Time Scaling (Link)

A deliberately small training recipe

The s1 study asks how little post-training is required to obtain controllable test-time scaling. It first gathers roughly 59,000 questions and teacher reasoning traces, then selects the s1K set of only 1,000 examples using three criteria: quality, difficulty, and domain diversity. Difficulty is estimated using failures of smaller models and reasoning length; diversity is enforced across about 50 domains. Supervised fine-tuning of Qwen2.5-32B-Instruct then minimizes

$$-\sum_t \log p_\theta(w_t \mid q, w_{<t})$$

on the distilled trace and answer. Training on all 59,000 candidates does not substantially beat the curated subset, and single-criterion selections perform worse. The result supports a data-selection claim, not the stronger claim that 1,000 examples suffice for a model without extensive pretraining and prior instruction tuning.

Budget forcing

Suppose a response format contains a delimiter separating hidden or explicit reasoning from the final answer. Budget forcing modifies decoding rather than model weights:

- to impose a maximum budget, insert the end-of-thinking delimiter once the reasoning reaches the token limit, forcing answer generation;
- to impose a minimum budget, suppress an early delimiter and append a continuation cue such as "Wait," inviting checking or an alternative derivation.

This is *sequential* test-time scaling: later tokens condition on and can revise earlier computation. It contrasts with parallel scaling, where one draws independent completions and selects or votes among them.

Let C denote the allowed reasoning-token budget and $A(C)$ the expected task accuracy under this decoding intervention. Test-time scaling is the empirical claim that $A(C)$ increases over a useful range of C; no theorem requires it to be monotone. Extra computation helps only when the model has learned useful revision behavior. It can instead amplify a false premise, wander, or replace a correct solution with an incorrect one.

Controls, results, and caveats

The authors compare budget forcing with verbal instructions to think longer, step-count conditioning, majority voting, and rejection sampling conditioned on length. Token-count

instructions are poorly obeyed, step targets can be met by changing step granularity rather than total computation, and selecting naturally long samples can show inverse scaling because long traces are often those that struggled. Directly intervening on the delimiter gives much tighter compute control. On AIME24, extending s1-32B's budget raises the reported score from 50 to 57 percent, although gains vary by benchmark and budget.

Budget forcing is best viewed as a causal decoding intervention, not proof that every emitted token is a useful logical operation. Results depend on the base model, the learned trace format, the continuation string, truncation policy, context length, and benchmark. Comparisons with closed systems are also sensitive to unequal compute and evaluation protocols.

10.4 Synthesis: Three Axes of Reasoning-System Scaling

The three papers scale different resources. DeepSeekMath improves the *training distribution* through mathematical and code data and introduces critic-free GRPO. DeepSeek-R1 scales *online rewarded search*, allowing a large policy to discover long solution strategies before converting them into usable and distillable assistant behavior. s1 concentrates *supervision* into a small curated set and scales *inference tokens* through a decoding intervention. Their results are complementary: test-time compute is valuable only when pretraining and post-training have produced a policy capable of using it, while training-time RL is valuable only to the extent that rewards and sampling expose meaningful differences among candidate trajectories.

11 Images, Generation, and Science

11.1 Generative Adversarial Nets (Link)

The adversarial game

Goodfellow et al. replace explicit likelihood evaluation with a two-player classification game. A generator maps noise $z \sim p_z$ to $G_\theta(z)$, thereby inducing a distribution p_g, while a discriminator $D_\phi(x) \in (0,1)$ estimates whether x came from the data. The idealized objective is

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}} \log D(x) + \mathbb{E}_{z\sim p_z} \log\Big(1 - D(G(z))\Big).$$

The discriminator supplies a learned discrepancy rather than a fixed pixel-space loss. Sampling needs only one forward pass through G; neither a Markov chain nor an explicit normalized density is required.

Proposition 11.1 (Optimal discriminator and population optimum)**.** *For a fixed generator with densities p_{data} and p_g, the pointwise optimal discriminator is*

$$D^*_G(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Substitution into the value function gives

$$C(G) := \max_D V(D,G) = -\log 4 + 2\,\mathrm{JS}\Big(p_{\text{data}} \,\|\, p_g\Big).$$

Consequently, the global minimum is $-\log 4$*, attained exactly when* $p_g = p_{\text{data}}$ *almost everywhere.*

The first statement follows by maximizing $a \log d + b \log(1 - d)$ independently at every x, whose stationary point is $d = a/(a + b)$. The second follows after writing the two logarithms relative to the mixture $m = (p_{\text{data}} + p_g)/2$. This is a population result over unrestricted functions; it does not imply convergence of alternating gradient descent with finite neural networks.

Optimization pathology

Early in training, a strong discriminator makes $D(G(z)) \approx 0$. The minimax generator loss $\log(1 - D(G(z)))$ can then saturate. The paper proposes the non-saturating alternative

$$\min_G -\mathbb{E}_{z \sim p_z} \log D(G(z)),$$

which has the same desired fixed point but stronger gradients. Instability, oscillation, and mode collapse arise because simultaneous learning makes the loss landscape nonstationary and because matching a distribution is not the same as covering every mode. The Jensen–Shannon theorem explains the ideal equilibrium, not the practical reliability of reaching it.

11.2 Denoising Diffusion Probabilistic Models (Link)

Forward corruption and exact Gaussian marginals

Diffusion models prescribe a fixed Markov chain that gradually destroys data:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(\sqrt{1 - \beta_t}\, x_{t-1}, \beta_t I\Big), \qquad \alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

Gaussian composition yields the crucial closed form

$$q(x_t \mid x_0) = \mathcal{N}\Big(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I\Big), \qquad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, I)$. Thus any noise level can be sampled in one step during training even though generation will traverse the reverse chain sequentially.

The exact posterior conditioned on the clean datum is also Gaussian,

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\Big(\widetilde{\mu}_t(x_t, x_0), \widetilde{\beta}_t I\Big), \qquad \widetilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

A neural reverse model approximates the unavailable posterior after x_0 is marginalized:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \qquad p(x_T) = \mathcal{N}(0, I).$$

Variational learning becomes noise prediction

Applying Jensen's inequality to $-\log p_\theta(x_0)$ gives a variational bound that decomposes into terminal, reverse-transition, and reconstruction terms. The intermediate terms are Gaussian KL divergences

$$\mathbb{E}_q \, \mathrm{KL}(q(x_{t-1} \mid x_t, x_0) \,|\, p_\theta(x_{t-1} \mid x_t)) \, .$$

Parameterizing the reverse mean through a predicted noise $\epsilon_\theta(x_t, t)$ reduces these terms, up to known time-dependent weights, to denoising regression. The paper's effective simplified objective is

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,x_0,\epsilon} \left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t \right) \right\|_2^2 .$$

Noise prediction is equivalent to estimating a scaled score $\nabla_{x_t} \log q_t(x_t)$, connecting diffusion to denoising score matching. Generation begins with Gaussian noise and repeatedly applies the learned reverse transition. The price of stable likelihood-based training and broad mode coverage is many sequential network evaluations.

11.3 High-Resolution Image Synthesis with Latent Diffusion Models (Link)

Moving diffusion away from pixels

Rombach et al. factor image generation into perceptual compression and semantic modeling. A pretrained autoencoder maps

$$z = \mathcal{E}(x), \qquad \widehat{x} = \mathcal{D}(z),$$

with a spatial downsampling factor f. A diffusion model is then trained on z rather than on all image pixels:

$$\mathcal{L}_{\text{LDM}} = \mathbb{E}_{z,\epsilon,t} \left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} z + \sqrt{1-\bar{\alpha}_t}\epsilon, t \right) \right\|_2^2 .$$

The autoencoder removes visually imperceptible detail and lowers spatial cost; the diffusion model allocates capacity to the remaining semantic and compositional variation. Too little compression preserves pixel-space cost, while too much imposes an irreversible reconstruction ceiling. The useful regime is therefore an empirical rate–distortion compromise.

General conditioning through cross-attention

A condition y—text, a semantic map, a class, or another modality—is encoded as a sequence $\tau_\varphi(y)$. At intermediate U-Net features h_i, cross-attention uses

$$Q = W_Q h_i, \qquad K = W_K \tau_\varphi(y), \qquad V = W_V \tau_\varphi(y),$$

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} \right) V.$$

Latent spatial positions can thereby retrieve different condition tokens. Unlike concatenating one global conditioning vector, this interface supports variable-length text and spatially

differentiated control. The model remains limited by the autoencoder, conditioning encoder, training distribution, and iterative sampler; latent diffusion reduces computation rather than making high-resolution synthesis free.

11.4 An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale (Link)

Images as token sequences

For an image $x \in \mathbb{R}^{H\times W\times C}$, divide it into non-overlapping $P \times P$ patches. Flattening gives $N = HW/P^2$ vectors $x_p^i \in \mathbb{R}^{P^2C}$. Vision Transformer (ViT) constructs

$$z_0 = \left[x_{\text{class}};\, x_p^1 E; \ldots; x_p^N E\right] + E_{\text{pos}},$$

then applies ordinary pre-normalized Transformer blocks,

$$z'_\ell = z_{\ell-1} + \text{MSA}(\text{LN}(z_{\ell-1})), \qquad z_\ell = z'_\ell + \text{MLP}(\text{LN}(z'_\ell)).$$

The final class-token representation feeds a classifier. Apart from patch extraction and positional embeddings, the architecture inserts little vision-specific structure.

Global self-attention gives every patch a one-layer path to every other patch, but costs $O(N^2)$. Smaller patches increase spatial resolution and rapidly increase attention cost. Compared with a CNN, ViT has weaker translation equivariance and locality biases; this hurts in smaller-data regimes but becomes less restrictive under large-scale pretraining. The paper's central empirical claim is consequently about scale and transfer: a generic Transformer can match or exceed strong convolutional models when pretrained on sufficiently large image collections, not that inductive bias is irrelevant.

11.5 Learning Transferable Visual Models from Natural Language Supervision (Link)

Symmetric image–text contrastive learning

CLIP learns from 400 million web image–text pairs rather than a fixed label vocabulary. Let normalized image and text embeddings in a batch of size B be u_i and v_i. With learned temperature $\tau > 0$, define logits

$$s_{ij} = \frac{u_i^\top v_j}{\tau}.$$

The symmetric contrastive loss is

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\left[-\frac{1}{B}\sum_i \log \frac{e^{s_{ii}}}{\sum_j e^{s_{ij}}} - \frac{1}{B}\sum_j \log \frac{e^{s_{jj}}}{\sum_i e^{s_{ij}}}\right].$$

The first term retrieves the paired text from each image; the second retrieves the paired image from each text. Other batch elements act as negatives, so batch composition and batch size affect the learning problem.

A classifier synthesized from language

For downstream class names $c_1, \ldots, c_K$, embed prompts such as "a photo of a c_k" and normalize them to obtain class vectors v_k. A new image with embedding u is classified by

$$p(c_k \mid x) = \frac{\exp(u^\top v_k / \tau)}{\sum_{j=1}^{K} \exp(u^\top v_j / \tau)}.$$

Thus natural language constructs the weights of a zero-shot linear classifier. Prompt ensembling averages or combines embeddings from multiple templates, reducing sensitivity to one phrasing.

Zero-shot transfer is not supervision-free: the web pairs provide massive, noisy supervision, and downstream class names encode task knowledge. CLIP matches the original supervised ResNet-50 on ImageNet without using ImageNet's training set, but performance varies widely across distribution shifts and specialized tasks. Dataset bias, duplicated test content, poor calibration, typographic attacks, and social stereotypes can all be inherited by the shared embedding space.

11.6 Highly Accurate Protein Structure Prediction with AlphaFold (Link)

Representations of evolutionary and geometric constraints

AlphaFold predicts a three-dimensional protein structure from an amino-acid sequence, using genetic database searches to construct a multiple sequence alignment (MSA) and, when available, structural templates. Its Evoformer maintains two coupled representations:

- an MSA representation indexed by sequence and residue, which captures conservation and correlated mutations;
- a pair representation indexed by residue pair (i, j), which stores hypotheses about their geometric relationship.

Row attention communicates along residues within homologous sequences; column attention communicates across homologues. An outer-product mean sends co-evolutionary information from the MSA into pair features. Triangle multiplication and triangle attention update (i, j) through intermediate residue k, reflecting the consistency constraints of geometry: relationships $i \leftrightarrow k$ and $k \leftrightarrow j$ inform $i \leftrightarrow j$.

Equivariant structure generation

The structure module assigns each residue a local rigid frame $T_i = (R_i, t_i) \in \mathrm{SE}(3)$ and predicts atomic coordinates in these frames. Invariant point attention computes attention using both learned features and distances between query and key points after transformation by their residue frames. Applying one common global rigid motion to all frames therefore

leaves the attention weights unchanged and moves the output structure equivariantly. The prediction cannot depend on an arbitrary laboratory coordinate system.

A central geometric loss is frame-aligned point error (FAPE). Schematically, for predicted and true frames T_i, T_i^* and atom positions x_j, x_j^*,

$$\mathcal{L}_{\text{FAPE}} = \frac{1}{N_F N_A} \sum_{i,j} \min\Big(d_{\max}, \left\| T_i^{-1} x_j - (T_i^*)^{-1} x_j^* \right\|_2\Big) .$$

Comparing coordinates in local frames makes the loss invariant to global rotation and translation while penalizing incorrect relative geometry. Auxiliary distogram, torsion, confidence, and stereochemical terms shape the learning signal. Recycling feeds predicted representations and structure back through the network for iterative refinement with shared parameters.

Accuracy, confidence, and scientific scope

On the blind CASP14 targets, AlphaFold attains near-experimental accuracy for a majority of structures and substantially exceeds competing methods. The model also predicts per-residue confidence (pLDDT); related aligned-error estimates help assess relative domain placement. These scores are estimates learned from data, not guarantees.

The system predicts a likely static structure under its modeling assumptions. It does not by itself solve protein dynamics, folding kinetics, ligand effects, all complexes, intrinsically disordered regions, or the causal mechanism of folding. Its achievement comes from combining evolutionary data, geometric inductive bias, end-to-end coordinate learning, and iterative refinement—not from replacing biology with a generic sequence model.

11.7 Synthesis: Representations Determine the Feasible Computation

The six papers can be read as choices of representation and supervision. GANs learn a discrepancy through a discriminator; diffusion models learn local Gaussian reversal; latent diffusion first compresses the space in which that reversal occurs. ViT represents images as patch sequences, while CLIP aligns visual and linguistic representations to make labels compositional. AlphaFold uses representations whose indices and transformations encode biological and Euclidean structure. Across all six, scaling succeeds when the representation makes the desired regularities accessible to the objective; neither model size nor a generic architecture removes the need for that match.

12 Under the Hood

12.1 Distilling the Knowledge in a Neural Network (Link)

Predictions as a transferable representation of knowledge

Hinton, Vinyals, and Dean distinguish the model used to discover structure from the model deployed under latency and memory constraints. Let teacher and student logits be v_i and z_i. At temperature T, their softened class probabilities are

$$q_i^{(T)} = \frac{e^{v_i/T}}{\sum_j e^{v_j/T}}, \qquad p_i^{(T)} = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

Large T exposes relative probabilities among incorrect classes—the teacher's "dark knowledge." For an image of a BMW, small probability on other cars and nearly zero probability on unrelated objects encodes similarity information that a one-hot label discards.

A typical student objective combines soft imitation with ordinary labels:

$$\mathcal{L} = \lambda T^2 H(q^{(T)}, p^{(T)}) + (1-\lambda) H(y, p^{(1)}).$$

The factor T^2 compensates for the approximately $1/T^2$ shrinkage of logit gradients at high temperature. When teacher logits are centered and T is large, matching soft probabilities approaches matching centered logits, showing why information exists beyond the winning class.

Distillation compresses a function, not its parameter values. The teacher can be an ensemble and the student a different architecture. Success depends on the transfer distribution: the student learns how the teacher behaves on the inputs shown to it, including the teacher's errors and biases. The paper also introduces specialist models for confusable class subsets, but modern usage most often refers to the temperature-based teacher–student objective.

12.2 Quantization and Training of Neural Networks (Link)

Affine integer representation

Jacob et al. represent a real tensor value r by an integer q through

$$r = S(q - Z),$$

where scale $S > 0$ and integer zero-point Z are chosen so that real zero is exactly representable. For a quantized matrix product, accumulation occurs in 32-bit integers:

$$a_j = \sum_k (q_{jk}^{(1)} - Z_1)(q_k^{(2)} - Z_2),$$

and the output is requantized approximately as

$$q_j^{(3)} = Z_3 + M a_j, \qquad M = \frac{S_1 S_2}{S_3},$$

with M implemented by a fixed-point multiplier and bit shift. Biases use the product scale $S_1 S_2$. These relations let the inference graph use integer arithmetic throughout rather than repeatedly dequantizing to floating point.

Quantization-aware training

Training inserts fake-quantization nodes that round and clip values in the forward pass while retaining floating-point parameters for optimization. If $Q(r)$ denotes the simulated quantizer, its true derivative is zero almost everywhere, so backpropagation uses a straight-through estimator,

$$\frac{\partial Q(r)}{\partial r} \approx \mathbf{1}\{r \text{ lies inside the unclipped range}\}.$$

Activation ranges are estimated during training, weights are quantized with fixed ranges, and batch-normalization parameters are folded into adjacent linear operations for inference. Quantization-aware training adapts weights to the errors actually induced by low precision and therefore generally preserves accuracy better than naive post-training rounding.

An 8-bit representation reduces storage and exploits efficient integer multiply–accumulate hardware, but theoretical operation counts alone do not determine latency. Kernel availability, accumulator width, memory layout, rescaling overhead, and device architecture matter. Accuracy may degrade for outliers or poorly calibrated activation ranges, and the straight-through gradient is an optimization heuristic rather than the derivative of rounding.

12.3 LoRA: Low-Rank Adaptation of Large Language Models (Link)

A low-dimensional update subspace

LoRA freezes a pretrained weight $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and parameterizes its task-specific change as

$$W = W_0 + \Delta W, \qquad \Delta W = BA, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r}, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}}).$$

For input x the layer becomes

$$h = W_0 x + \frac{\alpha}{r} BAx.$$

One factor is initialized randomly and the other to zero, so the initial model exactly equals the pretrained model. Only A and B receive optimizer state and gradients. The hypothesis is not that W_0 is low rank, but that the adaptation ΔW has low intrinsic rank.

The update can be merged into W_0 at deployment, producing no additional inference latency; it can also be unmerged to switch tasks. If LoRA is applied to attention projections, parameter cost falls from $d_{\text{out}} d_{\text{in}}$ per adapted matrix to $r(d_{\text{out}} + d_{\text{in}})$. The paper finds surprisingly small ranks sufficient in several language-model tasks and analyzes overlap among learned update subspaces.

Low trainable-parameter count is not identical to low total memory: the frozen base weights and activations remain, and training memory depends on the chosen layers and sequence length. Too small a rank constrains adaptation; large distribution shifts may require changes not captured by one low-rank update. Multiple adapters are compact but still require versioning, evaluation, and a decision about whether incompatible updates can safely be composed.

12.4 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness (Link)

The bottleneck is data movement

Standard attention computes

$$S = QK^\top, \qquad P = \mathrm{softmax}(S), \qquad O = PV.$$

Materializing the $N \times N$ matrices S and P in high-bandwidth memory (HBM) costs quadratic storage and repeated slow memory traffic. FlashAttention computes the same result by tiling Q, K, V into on-chip SRAM, fusing the operations, and never writing the full attention matrix to HBM. It changes the algorithm's memory schedule, not the mathematical attention rule.

Online softmax makes tiling exact

For one query row, suppose processed logits have maximum m and exponential sum $\ell = \sum_j e^{s_j - m}$. For a new block with maximum m_b, sum ℓ_b, and value-weighted numerator u_b, update

$$m' = \max(m, m_b), \qquad \ell' = e^{m-m'}\ell + e^{m_b-m'}\ell_b,$$

$$u' = e^{m-m'}u + e^{m_b-m'}u_b, \qquad O = \frac{u'}{\ell'}.$$

Rescaling by the new running maximum makes the recurrence algebraically equal to a single global softmax and numerically stable. Causal attention simply masks disallowed tiles or entries before this update.

With SRAM capacity M, the paper analyzes substantially fewer HBM accesses than standard materialization and proves optimality over a regime of M for the stated computational model. The backward pass stores normalization statistics and recomputes attention blocks rather than storing P; extra FLOPs are exchanged for much less IO. FlashAttention remains $O(N^2 d)$ arithmetic for dense attention. Its wall-clock gains arise because GPUs are often memory-bound, so FLOP count alone is a poor performance model.

12.5 Efficient Memory Management for LLM Serving with PagedAttention (Link)

Why the KV cache controls serving throughput

Autoregressive decoding caches every layer's key and value vectors for all previous tokens. For L layers and per-token key/value width d_{kv}, a sequence of length n requires memory proportional to

$$2Lnd_{\mathrm{kv}} \times \text{bytes per scalar}.$$

The cache grows one token at a time, request lengths are unknown, and beam or parallel sampling may share long prefixes. Reserving a contiguous maximum-size region wastes space

through internal fragmentation; repeated allocation and copying causes external fragmentation and duplication. Smaller feasible batches then leave compute capacity unused.

Virtual memory for attention

PagedAttention divides each request's logical KV sequence into fixed-size blocks. A block table maps logical block numbers to noncontiguous physical GPU blocks. The attention kernel follows this mapping while reading keys and values, analogously to address translation in an operating system. Physical blocks are allocated on demand, so waste is confined primarily to the final partially filled block rather than an entire maximum-length reservation.

Reference counts and copy-on-write allow beam search and parallel sampling to share prompt and prefix blocks. A physical block is copied only when one sequence modifies a shared logical block. This makes memory sharing safe while avoiding eager duplication. vLLM combines PagedAttention with continuous batching and scheduling, reporting two-to-fourfold throughput improvements at comparable latency in the paper's workloads.

PagedAttention does not reduce the asymptotic number of cached KV elements or the attention computation for a fixed sequence. It improves utilization and enables larger effective batches. Block size trades metadata and address overhead against fragmentation, while real performance depends on request lengths, arrival processes, decoding strategy, kernels, and hardware.

12.6 Switch Transformers: Scaling to Trillion Parameter Models (Link)

Sparse conditional computation

A Switch layer replaces a dense feed-forward sublayer with E expert networks. A router computes

$$p_e(x) = \frac{\exp((W_r x)_e)}{\sum_{j=1}^{E} \exp((W_r x)_j)}, \qquad e^*(x) = \arg\max_e p_e(x),$$

and sends each token to only its top-1 expert:

$$y = p_{e^*(x)}(x)\ \mathrm{FFN}_{e^*(x)}(x).$$

Total parameter count can grow with E while per-token expert computation remains close to that of one dense feed-forward network. Sparsity here means conditional activation, not that the expert weight matrices themselves contain mostly zeros.

Capacity and load balancing

Distributed experts have finite token capacity. With T tokens in a batch and capacity factor c, a typical per-expert limit is approximately

$$C = \left\lceil c \frac{T}{E} \right\rceil.$$

Tokens routed beyond capacity are dropped from the expert computation and pass through the residual path. Because an unconstrained router can collapse onto a few experts, the paper adds an auxiliary loss. If f_e is the fraction of tokens dispatched to expert e and P_e the average routing probability,

$$\mathcal{L}_{\text{aux}} = \alpha E \sum_{e=1}^{E} f_e P_e.$$

It is minimized near balanced traffic while remaining differentiable through P_e. Router computations are kept in higher precision, initialization is scaled down, and selective precision stabilizes otherwise fragile low-precision training.

Switch Transformers demonstrate that parameter capacity and FLOPs per token can be partly decoupled, producing large pretraining speedups and strong multilingual transfer. The costs move elsewhere: all expert parameters must be stored, tokens must be communicated across devices, load imbalance creates stragglers or dropped tokens, and fine-tuning can be unstable. A trillion parameters therefore does not mean a trillion parameters are evaluated for each token.

12.7 Synthesis: Six Different Meanings of Efficiency

These methods optimize different denominators. Distillation reduces the capacity required at deployment; quantization reduces bits and changes the available arithmetic; LoRA reduces trainable state per task; FlashAttention reduces memory traffic for one exact operator; PagedAttention reduces wasted dynamic serving memory; and Switch Transformers increase parameter capacity at roughly fixed active computation. Their gains can compose, but their headline numbers cannot be compared without specifying whether the constrained resource is parameters, optimizer memory, HBM traffic, KV-cache capacity, latency, throughput, energy, or statistical accuracy.